# 3 files and exceptions

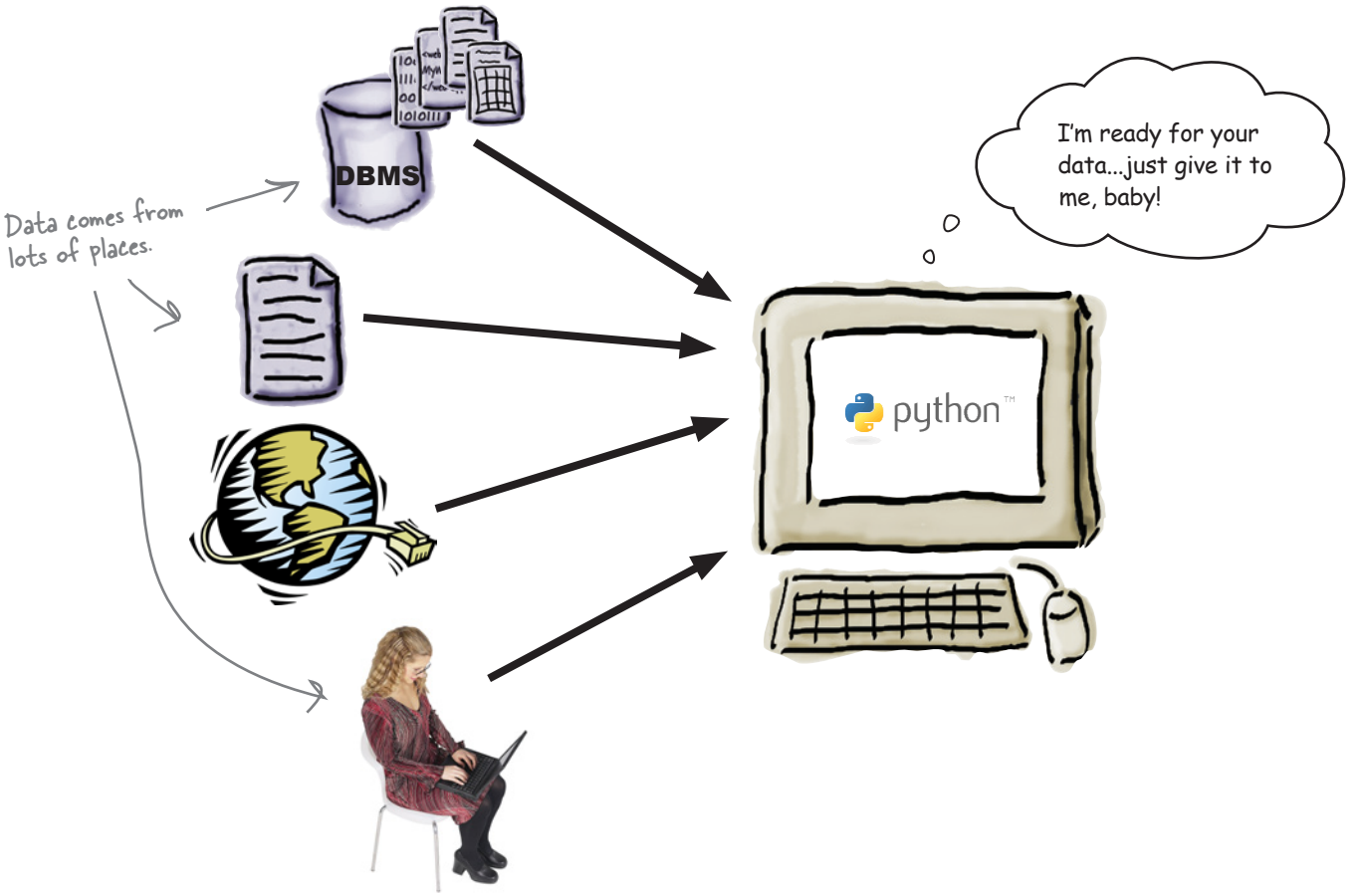
# Dealing with errors


**It's simply not enough to process your list data in your code.**

You need to be able to get your data *into* your programs with ease, too. It's no surprise then that Python makes reading data from **files** easy. Which is great, until you consider what can go *wrong* when interacting with data *external* to your programs…and there are lots of things waiting to trip you up! When bad stuff happens, you need a strategy for getting out of trouble, and one such strategy is to deal with any exceptional situations using Python's **exception handling** mechanism showcased in this chapter.

# Data is external to your program

Most of your programs conform to the *input-process-output model*: data comes in, gets manipulated, and then is stored, displayed, printed, or transferred.



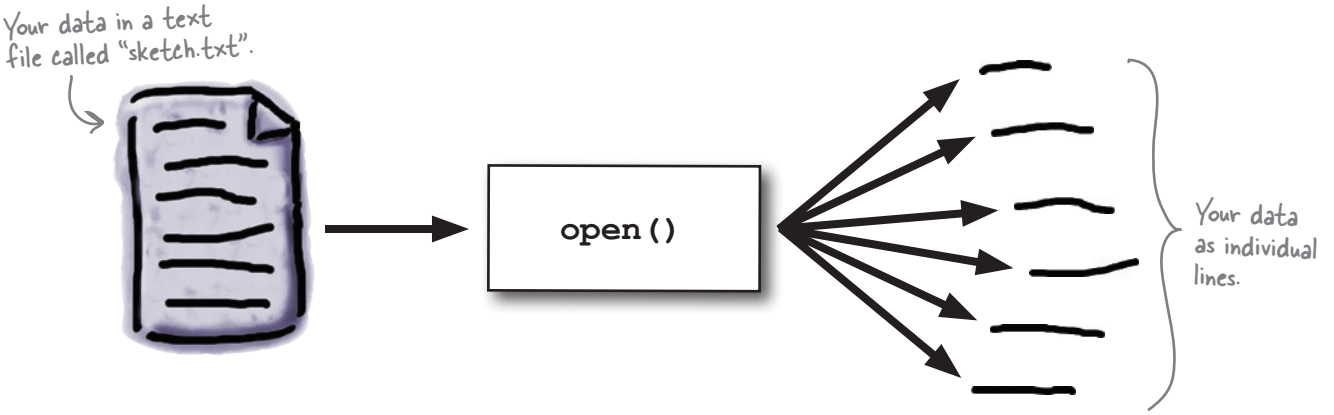

So far, you've learned how to **process** data as well as **display** it on screen. But what's involved in getting data into your programs? Specifically, what's involved in reading data from a file?

## How does Python read data from a file?

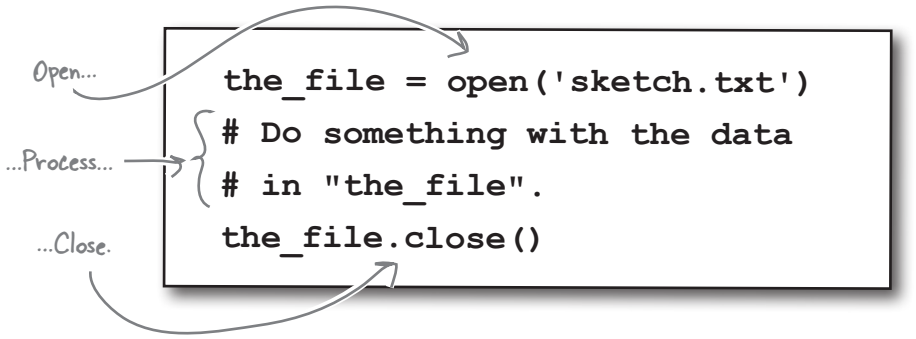
# It's all lines of text

The basic input mechanism in Python is **line based**: when read into your program from a text file, data arrives one line at a time.

Python's `open()` BIF lives to interact with files. When combined with a **for** statement, reading files is straightforward.




When you use the `open()` BIF to access your data in a file, an **iterator** is created to feed the lines of data from your file to your code one line at a time. But let's not get ahead of ourselves. For now, consider the standard *open-process-close* code in Python:





Create a folder called `HeadFirstPython` and a subfolder called `chapter3`. With the folders ready, download `sketch.txt` from the Head First Python support website and save it to the `chapter3` folder.

**Let's use IDLE to get a feel for Python's file-input mechanisms.**

## An IDLE Session

Start a new IDLE sesson and import the `os` module to change the current working directory to the folder that contains your just-downloaded data file:

```
>>> import os
>>> os.getcwd()
'/Users/barryp/Documents'
>>> os.chdir('../HeadFirstPython/chapter3')
>>> os.getcwd()
'/Users/barryp/HeadFirstPython/chapter3'
```

Import "os" from the Standard Library.

What's the current working directory?

Change to the folder that contains your data file.

Confirm you are now in the right place.

Now, open your data file and read the first two lines from the file, displaying them on screen:

```
>>> data = open('sketch.txt')
>>> print(data.readline(), end='')
Man: Is this the right room for an argument?
>>> print(data.readline(), end='')
Other Man: I've told you once.
```

Open a named file and assign the file to a file object called "data".

Use the "readline()" method to grab a line from the file, then use the "print()" BIF to display it on screen.

Let's "rewind" the file back to the start, then use a **for** statement to process every line in the file:

```
>>> data.seek(0)
0
>>> for each_line in data:
        print(each_line, end='')
Man: Is this the right room for an argument?
Other Man: I've told you once.
Man: No you haven't!
Other Man: Yes I have.
Man: When?
Other Man: Just now.
Man: No you didn't!
   ...
Man: (exasperated) Oh, this is futile!!
(pause)
Other Man: No it isn't!
Man: Yes it is!
>>> data.close()
```

Use the "seek()" method to return to the start of the file. And yes, you can use "tell()" with Python's files, too.

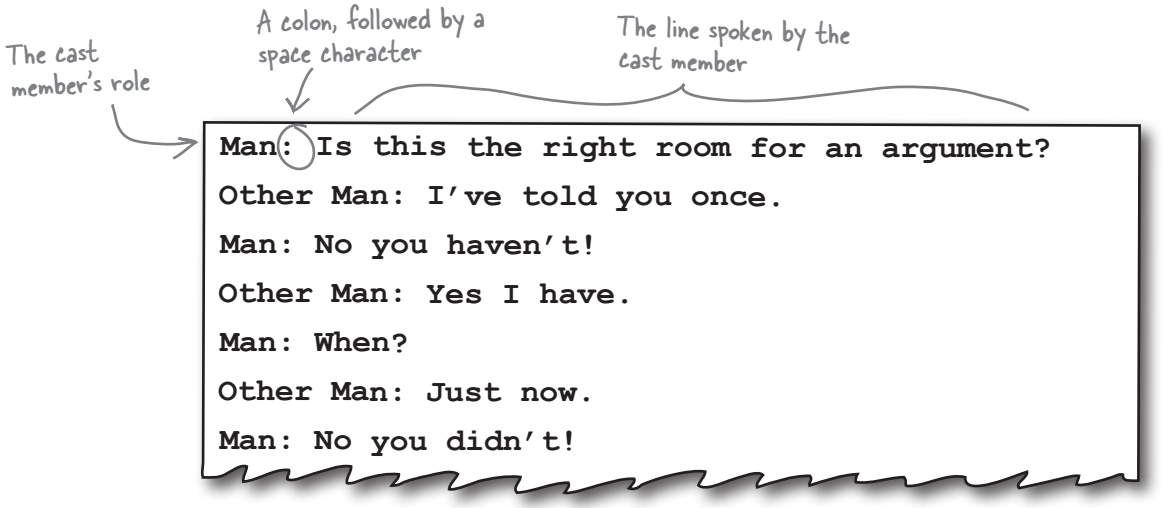
This code should look familiar: it's a standard iteration using the file's data as input.

Every line of the data is displayed on screen (although for space reasons, it is abridged here).

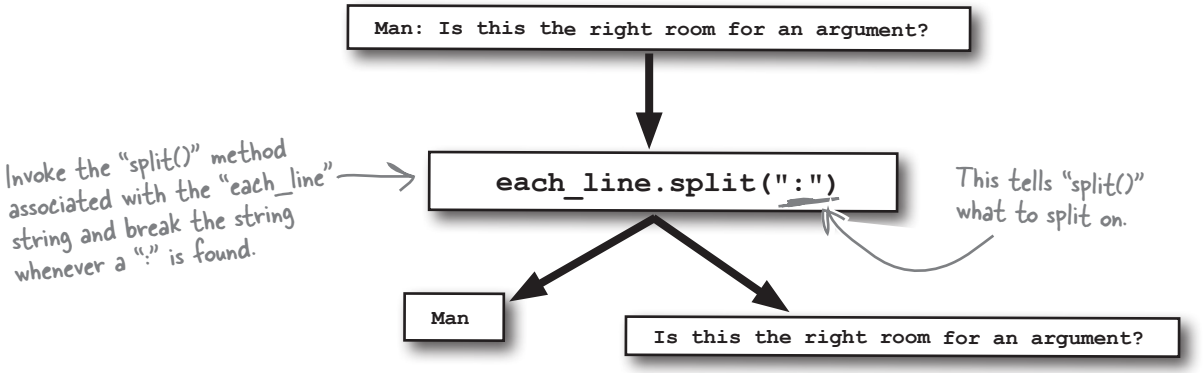
Since you are now done with the file, be sure to close it.

# Take a closer look at the data

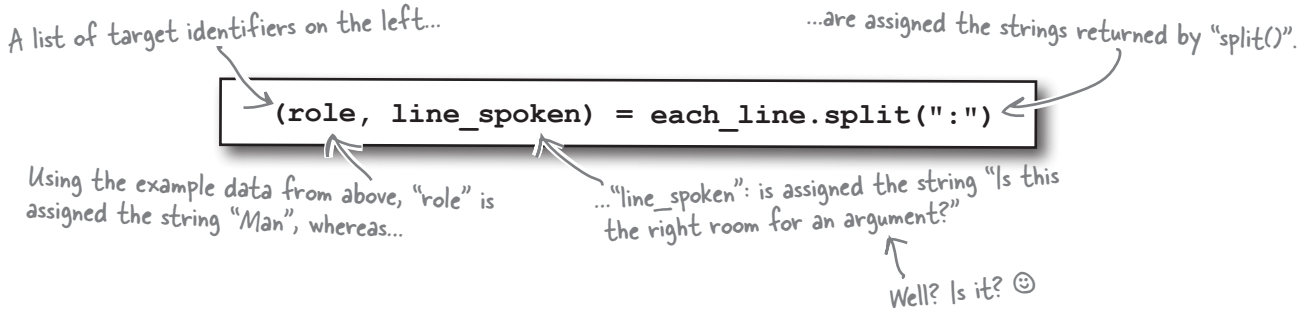
Look closely at the data. It appears to conform to a specific format:




With this format in mind, you can process each line to *extract* parts of the line as required. The `split()` method can help here:




The `split()` method returns a list of strings, which are assigned to a list of target identifiers. This is known as *multiple assignment*:



## An IDLE Session

Let's confirm that you can still process your file while splitting each line. Type the following code into IDLE's shell:

```
>>> data = open('sketch.txt')
>>> for each_line in data:
        (role, line_spoken) = each_line.split(':')
        print(role, end='')
        print(' said: ', end='')
        print(line_spoken, end='')
```

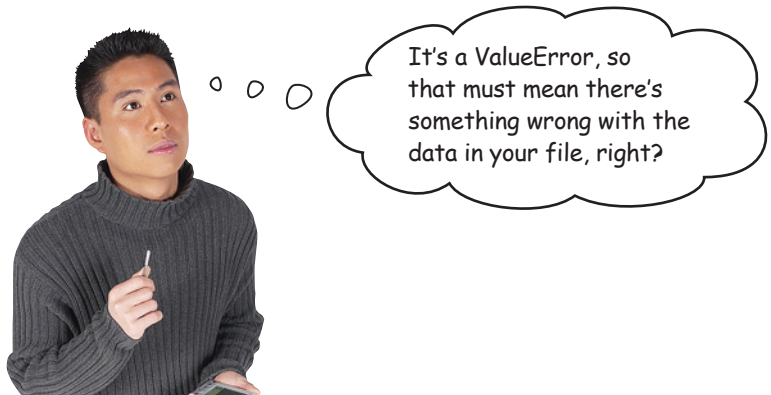
Open the data file.

Process the data, extracting each part from each line and displaying each part on screen.

```
Man said:  Is this the right room for an argument?
Other Man said:  I've told you once.
Man said:  No you haven't!
Other Man said:  Yes I have.
Man said:  When?
Other Man said:  Just now.
Man said:  No you didn't!
Other Man said:  Yes I did!
Man said:  You didn't!
Other Man said:  I'm telling you, I did!
Man said:  You did not!
Other Man said:  Oh I'm sorry, is this a five minute argument, or the full half hour?
Man said:  Ah! (taking out his wallet and paying) Just the five minutes.
Other Man said:  Just the five minutes. Thank you.
Other Man said:  Anyway, I did.
Man said:  You most certainly did not!
Traceback (most recent call last):
  File "<pyshell#10>", line 2, in <module>
    (role, line_spoken) = each_line.split(':')
ValueError: too many values to unpack
```

This all looks OK.

Whoops! There's something seriously wrong here.



# Know your data

Your code worked fine for a while, then crashed with a *runtime error*. The problem occurred right after the line of data that had the *Man* saying, "You most certainly did not!"

Let's look at the data file and see what comes after this successfully processed line:

```
Man: You didn't!
Other Man: I'm telling you, I did!
Man: You did not!
Other Man: Oh I'm sorry, is this a five minute argument, or the full half hour?
Man: Ah! (taking out his wallet and paying) Just the five minutes.
Other Man: Just the five minutes. Thank you.
Other Man: Anyway, I did.
Man: You most certainly did not!
Other Man: Now let's get one thing quite clear: I most definitely told you!
Man: Oh no you didn't!
Other Man: Oh yes I did!
```

The error occurs AFTER this line of data.

Notice anything?

Notice anything about the *next* line of data?

The next line of data has two colons, not one. This is enough extra data to upset the `split()` method due to the fact that, as your code currently stands, `split()` expects to break the line into two parts, assigning each to `role` and `line_spoken`, respectively.

When an extra colon appears in the data, the `split()` method breaks the line into *three parts*. Your code hasn't told `split()` what to do with the third part, so the Python interpreter *raises* a `ValueError`, complains that you have "too many values," and terminates. A **runtime error** has occurred.


What approach might you take to solve this data-processing problem?


To help diagnose this problem, let's put your code into its own file called `sketch.py`. You can copy and paste your code from the IDLE shell into a new IDLE edit window.

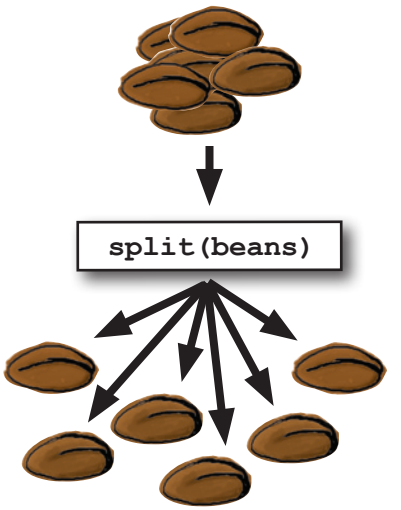
# Know your methods and ask for help

It might be useful to see if the `split()` method includes any functionality that might help here. You can ask the IDLE shell to tell you more about the `split()` method by using the `help()` BIF.

## An IDLE Session

```
>>> help(each_line.split)
Help on built-in function split:

split(...)
    S.split([sep[, maxsplit]]) -> list of strings

    Return a list of the words in S, using sep as the
    delimiter string.  If maxsplit is given, at most maxsplit
    splits are done. If sep is not specified or is None, any
    whitespace string is a separator and empty strings are
    removed from the result.
```

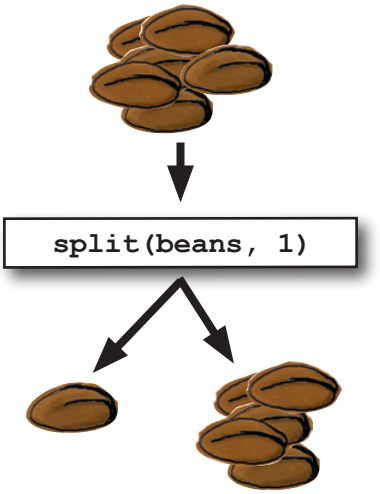
Looks like "split()" takes an optional argument.




The optional argument to `split()` controls how many breaks occur within your line of data. By default, the data is broken into as many parts as is possible. But you need only two parts: the name of the character and the line he spoke.

If you set this optional argument to `1`, your line of data is only ever broken into two pieces, effectively negating the effect of any extra colon on any line.

**Let's try this and see what happens.**




## Geek Bits

IDLE gives you searchable access to the entire Python documentation set via its Help → Python Docs menu option (which will open the docs in your web browser). If all you need to see is the documentation associated with a single method or function, use the **`help()`** BIF within IDLE's shell.

## An IDLE Session

Here's the code in the IDLE edit window. Note the extra argument to the `split()` method.

sketch.py – /Users/barryp/HeadFirstPython/chapter4/sketch.py

```
data = open('sketch.txt')

for each_line in data:
    (role, line_spoken) = each_line.split(':', 1)
    print(role, end='')
    print(' said: ', end='')
    print(line_spoken, end='')

data.close()
```

Ln: 11 Col: 0

The extra argument controls how "split()" splits.

With the edit applied and saved, press F5 (or select **Run Module** from IDLE's **Run** menu) to try out this version of your code:

```
>>> ================================ RESTART ================================
>>>
Man said:  Is this the right room for an argument?
Other Man said:  I've told you once.
Man said:  No you haven't!
Other Man said:  Yes I have.
Man said:  When?
Other Man said:  Just now.
    ...
Other Man said:  Anyway, I did.
Man said:  You most certainly did not!
Other Man said:  Now let's get one thing quite clear: I most definitely told you!
Man said:  Oh no you didn't!
Other Man said:  Oh yes I did!
Man said:  Oh no you didn't!
Other Man said:  Oh yes I did!
Man said:  Oh look, this isn't an argument!
Traceback (most recent call last):
  File "/Users/barryp/HeadFirstPython/chapter4/sketch.py", line 5, in <module>
    (role, line_spoken) = each_line.split(':', 1)
ValueError: need more than 1 value to unpack
```

The displayed output is abridged to allow the important stuff to fit on this page.

Cool. You made it past the line with two colons...

...but your joy is short lived. There's ANOTHER ValueError!!

**That's enough to ruin your day. What could be wrong now?**

# Know your data (better)

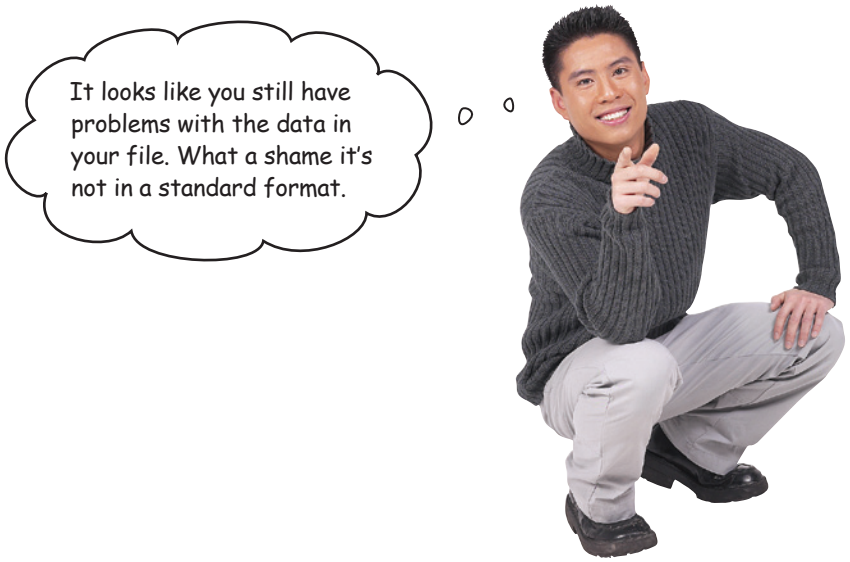
Your code has raised another `ValueError`, but this time, instead of complaining that there are "too many values," the Python interpreter is complaining that it doesn't have enough data to work with: "need more than 1 value to unpack." Hopefully, another quick look at the data will clear up the mystery of the missing data.

```
Other Man: Now let's get one thing quite clear: I most definitely told you!
Man: Oh no you didn't!
Other Man: Oh yes I did!
Man: Oh no you didn't!
Other Man: Oh yes I did!
Man: Oh look, this isn't an argument!
(pause)
Other Man: Yes it is!
Man: No it isn't!
(pause)
Man: It's just contradiction!
Other Man: No it isn't!
```

What's this?!? Some of the data doesn't conform to the expected format...which can't be good.

## The case of the missing colon

Some of the lines of data contain no colon, which causes a problem when the `split()` method goes looking for it. The lack of a colon prevents `split()` from doing its job, causes the runtime error, which then results in the complaint that the interpreter needs "more than 1 value."



# Two very different approaches


Jill's suggested approach certainly works: add the extra logic required to work out whether it's worth invoking `split()` on the line of data. All you need to do is work out how to check the line of data.

Joe's approach works, too: let the error occur, spot that it has happened, and then recover from the runtime error…somehow.

**Which approach works best here?**

# Add extra logic

Let's try *each* approach, then decide which works best here.

In addition to the `split()` method, every Python string has the `find()` method, too. You can ask `find()` to try and locate a substring in another string, and if it can't be found, the `find()` method returns the value `-1`. If the method locates the substring, `find()` returns the index position of the substring in the string.

## An IDLE Session

Assign a string to the `each_line` variable that does not contain a colon, and then use the `find()` method to try and locate a colon:

```
>>> each_line = "I tell you, there's no such thing as a flying circus."
>>> each_line.find(':')
-1
```

The string does NOT contain a colon, so "find()" returns –1 for NOT FOUND.

Press **Alt-P** *twice* to recall the line of code that assigns the string to the variable, but this time edit the string to include a colon, then use the `find()` method to try to locate the colon:

```
>>> each_line = "I tell you: there's no such thing as a flying circus."
>>> each_line.find(':')
10
```

The string DOES contain a colon, so "find()" returns a positive index value.


Adjust your code to use the extra logic technique demonstrated on the previous page to deal with lines that don't contain a colon character.

```
data = open('sketch.txt')

for each_line in data:
    if ..........................................................................
        (role, line_spoken) = each_line.split(':', 1)
        print(role, end='')
        print(' said: ', end='')
        print(line_spoken, end='')

data.close()
```

What condition needs to go here?


Can you think of any potential problems with this technique? Grab your pencil and write down any issues you might have with this approach in the space provided below:

## Exercise Solution

You were to adjust your code to use the extra logic technique to deal with lines that don't contain a colon character:

```
data = open('sketch.txt')

for each_line in data:
    if not each_line.find(':') == -1:
        (role, line_spoken) = each_line.split(':', 1)
        print(role, end='')
        print(' said: ', end='')
        print(line_spoken, end='')

data.close()
```

It takes a few seconds to get your head around this condition, but it does work.

Note the use of the "not" keyword, which negates the value of the condition.

## Sharpen your pencil Solution

You were to think of any potential problems with this technique, grabbing your pencil to write down any issues you might have with this approach.

It's OK if your issues are different. Just so long as they are similar to these.

There might be a problem with this code if the format of the data file changes, which will require changes to the condition.

The condition used by the if statement is somewhat hard to read and understand.

This code is a little "fragile"...it will break if another exceptional situation arises.

# Test Drive

Amend your code within IDLE's edit window, and press F5 to see if it works.

Python Shell

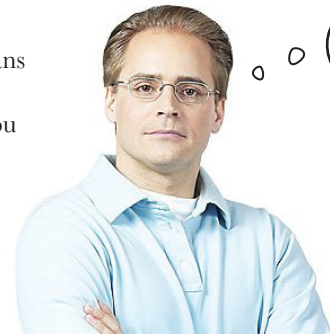
```
>>> ================================ RESTART ================================
>>>
Man said:  Is this the right room for an argument?
Other Man said:  I've told you once.
Man said:  No you haven't!
Other Man said:  Yes I have.
Man said:  When?
Other Man said:  Just now.
Man said:  No you didn't!
Other Man said:  Yes I did!
Man said:  You didn't!
Other Man said:  I'm telling you, I did!
Man said:  You did not!
Other Man said:  Oh I'm sorry, is this a five minute argument, or the full half hour?
Man said:  Ah! (taking out his wallet and paying) Just the five minutes.
Other Man said:  Just the five minutes. Thank you.
Other Man said:  Anyway, I did.
Man said:  You most certainly did not!
Other Man said:  Now let's get one thing quite clear: I most definitely told you!
Man said:  Oh no you didn't!
Other Man said:  Oh yes I did!
Man said:  Oh no you didn't!
Other Man said:  Oh yes I did!
Man said:  Oh look, this isn't an argument!
Other Man said:  Yes it is!
Man said:  No it isn't!
Man said:  It's just contradiction!
Other Man said:  No it isn't!
Man said:  It IS!
Other Man said:  It is NOT!
Man said:  You just contradicted me!
Other Man said:  No I didn't!
Man said:  You DID!
Other Man said:  No no no!
Man said:  You did just then!
Other Man said:  Nonsense!
Man said:  (exasperated) Oh, this is futile!!
Other Man said:  No it isn't!
Man said:  Yes it is!
>>>
```

Ln: 149 Col: 4

No errors this time.

Your program works…although it is **fragile**.

If the format of the file changes, your code will need to change, too, and *more code* generally means *more complexity*. Adding extra logic to handle exceptional situations works, but it might cost you in the long run.



Maybe it's time for a different approach? One that doesn't require extra logic, eh?

# Handle exceptions

Have you noticed that when something goes wrong with your code, the Python interpreter displays a *traceback* followed by an error message?

The **traceback** is Python's way of telling you that something *unexpected* has occurred during runtime. In the Python world, runtime errors are called **exceptions**.


Of course, if you decide to *ignore* an exception when it occurs, your program crashes and burns.

But here's the skinny: Python let's you *catch* exceptions as they occur, which gives you with a chance to possibly recover from the error and, critically, **not** crash.

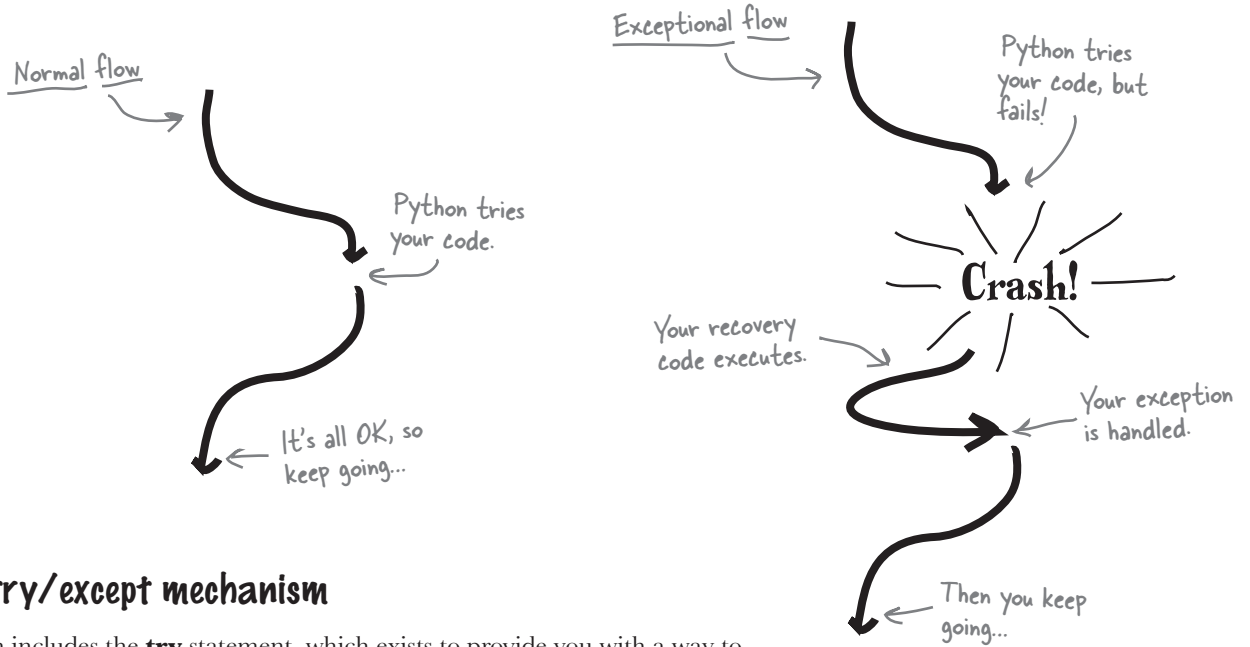
By controlling the runtime behavior of your program, you can ensure (as much as possible) that your Python programs are robust in the face of *most* runtime errors.

**Try the code first. Then deal with errors as they happen.**

# Try first, then recover

Rather than adding extra code and logic to guard against bad things happening, Python's **exception handling** mechanism lets the error occur, spots that it has happened, and then gives you an opportunity to recover.

During the *normal flow of control*, Python tries your code and, if nothing goes wrong, your code continues as normal. During the *exceptional flow of control*, Python tries your code only to have something go wrong, your recovery code executes, and then your code continues as normal.




## The try/except mechanism

Python includes the **try** statement, which exists to provide you with a way to systematically handle exceptions and errors at runtime. The general form of the **try** statement looks like this:

Both "try" and "except" are Python keywords.

```
try:
    your code (which might cause a runtime error)
except:
    your error-recovery code
```


**No. Not mad. And, yes. Letting errors occur.**

If you try to code for every possible error, you'll be at it for a long time, because all that extra logic takes a while to work out.

Paradoxically, when you worry less about covering *every* possible error condition, your coding task actually gets *easier*.

# Identify the code to protect

In order to plug into the Python exception handling mechanism, take a moment to identify the code that you need to *protect*.

## Sharpen your pencil

Study your program and circle the line or lines of code that you think you need to protect. Then, in the space provided, state why.

```
data = open('sketch.txt')

for each_line in data:
        (role, line_spoken) = each_line.split(':', 1)
        print(role, end='')
        print(' said: ', end='')
        print(line_spoken, end='')

data.close()
```

State your reason why here.

.....

.....

.....

.....

.....

## there are no Dumb Questions

**Q: Something has been bugging me for a while. When the split() method executes, it passes back a list, but the target identifiers are enclosed in regular brackets, not square brackets, so how is this a list?**

A: Well spotted. It turns out that there are **two** types of list in Python: those that can change (enclosed in square brackets) and those that cannot be changed once they have been created (enclosed in regular brackets). The latter is an *immutable* list, more commonly referred to as a *tuple*. Think of tuples as the same as a list, except for one thing: once created, the data they hold **cannot** be changed under any circumstances. Another way to think about tuples is to consider them to be a *constant list*. At Head First, we pronounce "tuple" to rhyme with "couple." Others pronounce "tuple" to rhyme with "rupal." There is no clear concensus as to which is correct, so pick one and stick to it.

## Sharpen your pencil Solution

You were to study your program and circle the line or lines of code that you think you need to protect. Then, in the space provided, you were to state why.

```
data = open('sketch.txt')

for each_line in data:
        (role, line_spoken) = each_line.split(':', 1)
        print(role, end='')
        print(' said: ', end='')
        print(line_spoken, end='')

data.close()
```

These four lines of code all need to be protected.

If the call to "split()" fails, you don't want the three "print()" statements executing, so it's best to protect all four lines of the "if" suite, not just the line of code that calls "split()".

# Take a pass on the error

With this data (and this program), it is best if you ignore lines that don't conform to the expected format. If the call to the `split()` method causes an exception, let's simply **pass** on reporting it as an error.

When you have a situation where you might be expected to provide code, but don't need to, use Python's `pass` statement (which you can think of as the *empty* or *null* statement.)

Here's the `pass` statement combined with `try`:

This code is protected from runtime errors.

```
data = open('sketch.txt')

for each_line in data:
    try:
        (role, line_spoken) = each_line.split(':', 1)
        print(role, end='')
        print(' said: ', end='')
        print(line_spoken, end='')
    except:
        pass
data.close()
```

If a runtime error occurs, this code is executed.

Now, no matter what happens when the `split()` method is invoked, the `try` statement *catches* any and all exceptions and *handles* them by ignoring the error with `pass`.

**Let's see this code in action.**


Make the required changes to your code in the IDLE edit window.

## An IDLE Session

With your code in the IDLE edit window, press F5 to run it.

sketch-try.py - /Users/barryp/HeadFirstPython/chapter4/sketch-try.py

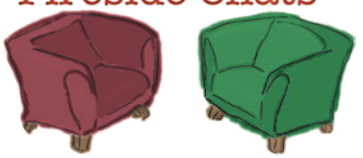
```
data = open('sketch.txt')

for each_line in data:
    try:
        (role, line_spoken) = each_line.split(':', 1)
        print(role, end='')
        print(' said: ', end='')
        print(line_spoken, end='')
    except:
        pass

data.close()
```

Ln: 14 Col: 0

```
>>> ================================ RESTART ================================
>>>
Man said:  Is this the right room for an argument?
Other Man said:  I've told you once.
Man said:  No you haven't!
Other Man said:  Yes I have.
Man said:  When?
    ...
OOther Man said:  Nonsense!
Man said:  (exasperated) Oh, this is futile!!
Other Man said:  No it isn't!
Man said:  Yes it is!
```

This code works, and there are no runtime errors, either.

# Fireside Chats



Tonight's talk: **Approaching runtime errors with extra code and exception handlers**

**Extra Code:**

By making sure runtime errors never happen, I keep my code safe from tracebacks.

Complexity never hurt anyone.

I just don't get it. You're more than happy for your code to explode in your face…then you decide it's probably a good idea to put out the fire?!?

But the bad things *still* happen to you. They never happen with me, because I don't let them.

Well…that depends. If you're smart enough—and, believe me, I am—you can think up all the possible runtime problems and code around them.

Hard work never hurt anyone.

Of course all my code is needed! How else can you code around all the runtime errors that are going to happen?

Um, uh…most of them, I guess.

Look: just cut it out. OK?

**Exception Handler:**

At the cost of added complexity….

I'll be sure to remind you of that the next time you're debugging a complex piece of code at 4 o'clock in the morning.

Yes. I concentrate on getting my work done first and foremost. If bad things happen, I'm ready for them.

Until something else happens that you weren't expecting. Then you're toast.

Sounds like a whole heap of extra work to me.

You did hear me earlier about debugging at 4 AM, right? Sometimes I think you actually enjoy writing code that you don't need…

Yeah…how many?

You don't know, do you? You've no idea what will happen when an unknown or unexpected runtime error occurs, do you?

# What about other errors?

It is true that both approaches work, but let's consider what happens when other errors surface.


## Handling missing files

Frank's posed an interesting question and, sure enough, the problem caused by the removal of the data file makes life more complex for Jill and Joe. When the data file is missing, *both* versions of the program crash with an `IOError`.


Rename the data file, then run both versions of your program again to confirm that they do indeed raise an `IOError` and generate a traceback.

# Add more error-checking code...

If you're a fan of the "let's not let errors happen" school of thinking, your first reaction will be to *add extra code* to check to see if the data file exists before you try to open it, right?

Let's implement this strategy. Python's `os` module has some facilities that can help determine whether a data file exists, so we need to import it from the Standard Library, then add the required check to the code:

Check whether the file exists.

All of this code remains unchanged.

Inform the user of the bad news.

```
import os

if os.path.exists('sketch.txt'):
    data = open('sketch.txt')

    for each_line in data:
        if not each_line.find(':') == -1:
            (role, line_spoken) = each_line.split(':', 1)
            print(role, end='')
            print(' said: ', end='')
            print(line_spoken, end='')

    data.close()
else:
    print('The data file is missing!')
```

## An IDLE Session

A quick test of the code confirms that this new problem is dealt with properly. With this new version of your code in IDLE's edit window, press F5 to confirm all is OK.

```
>>> ================================ RESTART ================================
>>>
The data file is missing!
>>>
```

Exactly what was expected. Cool.

# ...Or add another level of exception handling

If you are a fan of the "handle exceptions as they occur" school of thinking, you'll simply wrap your code within another **try** statement.

Add another "try" statement.

As with the other program, all of this code remains unchanged.

Give the user the bad news.

```
try:
    data = open('sketch.txt')

    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)
            print(role, end='')
            print(' said: ', end='')
            print(line_spoken, end='')
        except:
            pass

    data.close()
except:
    print('The data file is missing!')
```

## An IDLE Session

Another quick test is required, this time with the version of your program that uses exception handling. Press F5 to give it a spin.

```
>>> ================================ RESTART ================================
>>>
The data file is missing!
>>>
```

As expected, this version of the program handles the missing file, too.

# So, which approach is best?

Well…it depends on who you ask! Here are both versions of your code:

sketch-extra-logic-file-missing.py – /Users/barryp/HeadFirstPython/chapter3/sk...

```
import os

if os.path.exists('sketch.txt'):
    data = open('sketch.txt')

    for each_line in data:
        if not each_line.find(':') == -1:
            (role, line_spoken) = each_line.split(':', 1)
            print(role, end='')
            print(' said ', end='')
            print(line_spoken, end='')

    data.close()
else:
    print('The datafile is missing!')
```

This version uses extra logic to handle File I/O errors.

sketch-with-try-file-missing.py – /Users/barryp/HeadFirstPython/chapter3/sket...

```
try:
    data = open('sketch.txt')

    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)
            print(role, end='')
            print(' said ', end='')
            print(line_spoken, end='')
        except:
            pass

    data.close()
except:
    print('The datafile is missing!')
```

Ln: 17 Col: 0

This version uses another "try" statement to handle File I/O errors.

Let's ask a simple question about these two versions of your program: *What do each of these programs do?*

## Sharpen your pencil

Grab your pencil. In box 1, write down what you think the program on the left does. In box 2, write down what you think the program on the right does.

1

.....

.....

.....

2

.....

.....

.....

## Sharpen your pencil Solution

You were to grab your pencil, then in box 1, write down what you thought the program on the left does. In box 2, write down what you thought the program on the right does.

There's a lot to write, so you actually need more space for your description than was provided on the previous page.

1. The code on the right starts by importing the "os" library, and then it uses "path.exists" to make sure the data file exists, before it attempts to open the data file. Each line from the file is then processed, but only after it has determined that the line conforms to the required format by checking first for a single ":" character in the line. If the ":" is found, the line is processed; otherwise, it's ignored. When we're all done, the data file is closed. And you get a friendly message at the end if the file is not found.

Now...that's more like it.

2. The code on the right opens a data file, processes each line in that file, extracts the data of interest and displays it on screen. The file is closed when done. If any exceptions occur, this code handles them.

## Complexity is rarely a good thing

Do you see what's happening here?

As the list of errors that you have to worry about grows, the **complexity** of the "add extra code and logic" solution increases to the point where it starts to *obscure the actual purpose of the program*.

This is not the case with the exceptions handling solution, in which it's *obvious* what the main purpose of the program is.

By using Python's exception-handling mechanism, you get to concentrate on what your code *needs to do*, as opposed to worrying about what can go wrong and writing extra code to avoid runtime errors.

Prudent use of the `try` statement leads to code that is easier to read, easier to write, and—perhaps most important—easier to fix when something goes wrong.

**Concentrate on what your code needs to do.**

# You're done...except for one small thing

Your exception-handling code is good. In fact, your code might be *too good* in that it is *too general*.

At the moment, no matter what error occurs at runtime, it is handled by your code because it's *ignored* or a error message is *displayed*. But you really need to worry only about `IOErrors` and `ValueErrors`, because those are the types of exceptions that occurred earlier when your were developing your program.

Although it is great to be able to handle all runtime errors, it's probably unwise to be too generic…you will want to know if something other than an `IOError` or `ValueError` occurs as a result of your code executing at runtime. If something else does happen, your code might be handling it in an *inappropriate* way.

```
try:
    data = open('sketch.txt')

    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)
            print(role, end='')
            print(' said: ', end='')
            print(line_spoken, end='')
        except:
            pass

    data.close()
except:
    print('The data file is missing!')
```

This code and this code runs when ANY runtime error occurs within the code that is being tried.

As your code is currently written, it is too generic. Any runtime error that occurs is handled by one of the `except` suites. This is unlikely to be what you want, because this code has the potential to *silently ignore runtime errors*.

**You need to somehow use except in a less generic way.**

# Be specific with your exceptions

If your exception-handling code is designed to deal with a specific type of error, be sure to specify the error type on the `except` line. In doing so, you'll take your exception handling code from *generic* to *specific*.

```
try:
      data = open('sketch.txt')

      for each_line in data:
          try:
              (role, line_spoken) = each_line.split(':', 1)
              print(role, end='')
              print(' said: ', end='')
              print(line_spoken, end='')
          except ValueError:
              pass
      data.close()
except IOError:
      print('The data file is missing!')
```

Specify the type of runtime error you are handling.

Of course, if an *different* type of runtime error occurs, it is no longer handled by your code, but at least now you'll get to hear about it. When you are specific about the runtime errors your code handles, your programs no longer silently ignore some runtime errors.

# Your Python Toolbox

**You've got Chapter 3 under your belt and you've added some key Python techiques to your toolbox.**

## Python Lingo

- An "exception" occurs as a result of a runtime error, producing a traceback.
- A "traceback" is a detailed description of the runtime error that has occurred.

## IDLE Notes

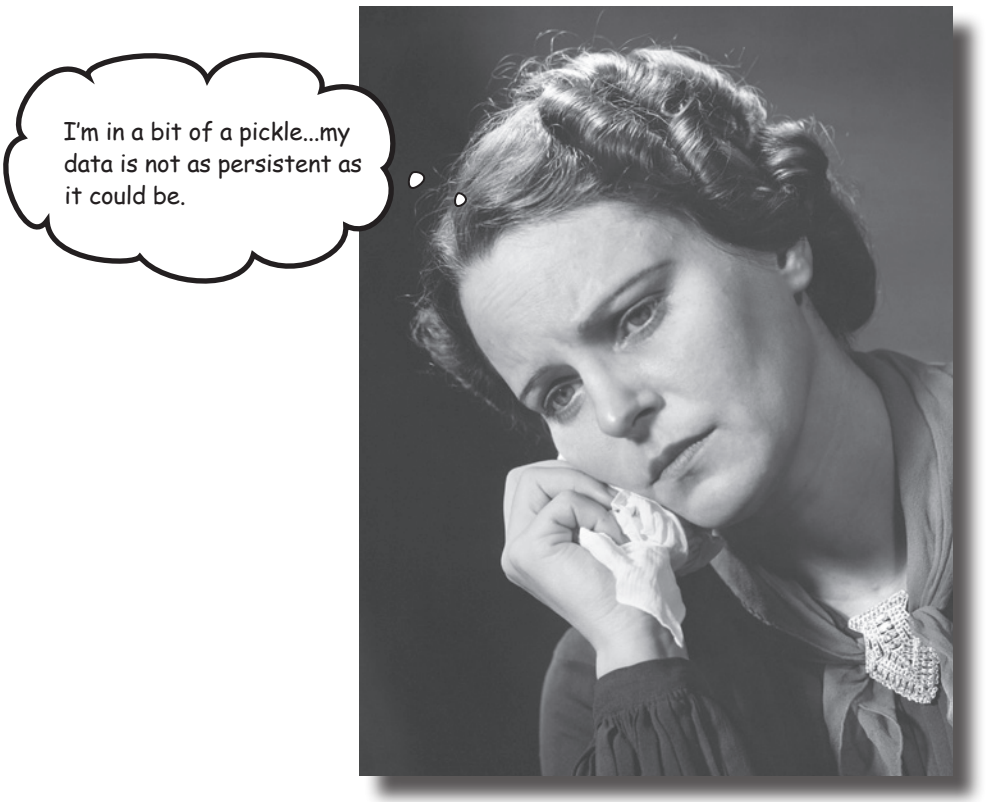
- Access Python's documentation by choosing Python Docs from IDLE's Help menu. The Python 3 documentation set should open in your favorite web browser.

## BULLET POINTS

- Use the `open()` BIF to open a disk file, creating an iterator that reads data from the file one line at a time.
- The `readline()` method reads a single line from an opened file.
- The `seek()` method can be used to "rewind" a file to the beginning.
- The `close()` method closes a previously opened file.
- The `split()` method can break a string into a list of parts.
- An unchangeable, constant list in Python is called a **tuple**. Once list data is assigned to a tuple, it cannot be changed. Tuples are *immutable*.
- A `ValueError` occurs when your data does not conform to an expected format.
- An `IOError` occurs when your data cannot be accessed properly (e.g., perhaps your data file has been moved or renamed).
- The `help()` BIF provides access to Python's documentation within the IDLE shell.
- The `find()` method locates a specific substring within another string.
- The `not` keyword negates a condition.
- The `try`/`except` statement provides an exception-handling mechanism, allowing you to protect lines of code that might result in a runtime error.
- The `pass` statement is Python's empty or null statement; it does nothing.

# 4 persistence

# Saving data to files

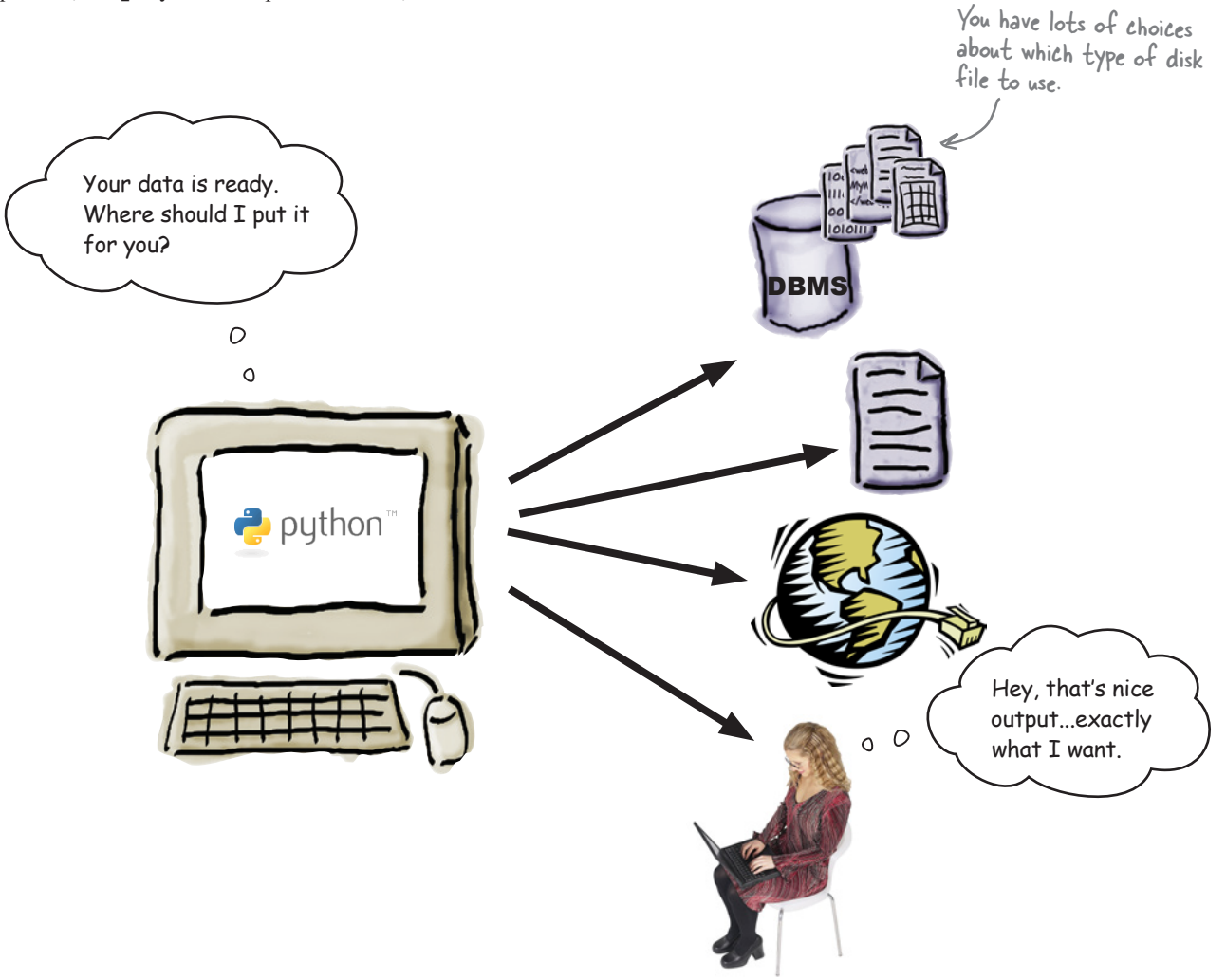



## It is truly great to be able to process your file-based data.

But what happens to your data when you're done? Of course, it's best to save your data to a disk file, which allows you to use it again at some later date and time. Taking your memory-based data and storing it to disk is what **persistence** is all about. Python supports all the usual tools for writing to files and also provides some cool facilities for *efficiently* storing Python data. So…flip the page and let's get started learning them.

# Programs produce data

It's a rare program that reads data from a disk file, processes the data, and then throws away the processed data. Typically, programs **save** the data they process, **display** their output on screen, or **transfer** data over a network.




Before you learn what's involved in writing data to disk, let's process the data from the previous chapter to work out who said what to whom.

**When that's done, you'll have something worth saving.**

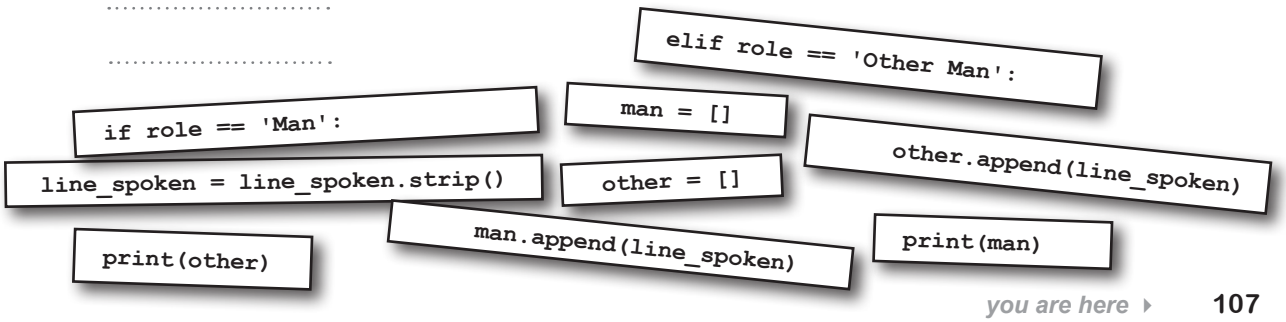
# Code Magnets

Add the code magnets at the bottom of this page to your existing code to satisfy the following requirements:

1. Create an empty list called `man`.
2. Create an empty list called `other`.
3. Add a line of code to remove unwanted whitespace from the `line_spoken` variable.
4. Provide the conditions and code to add `line_spoken` to the correct list based on the value of `role`.
5. Print each of the lists (`man` and `other`) to the screen.

```
.....

.....
try:
    data = open('sketch.txt')
    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)

            ..........................................................................

            ..................................

                ..................................................

            ..................................

                ..................................................

        except ValueError:
            pass
    data.close()
except IOError:
    print('The datafile is missing!')

.....

.....
```

Here are your magnets.



# Code Magnets Solution

Your were to add the code magnets to your existing code to satisfy the following requirements:

1. Create an empty list called `man`.
2. Create an empty list called `other`.
3. Add a line of code to remove unwanted whitespace from the `line_spoken` variable.
4. Provide the conditions and code to add `line_spoken` to the correct list based on the value of `role`.
5. Print each of the lists (`man` and `other`) to the screen.

Assign an empty list to "man" and "other".

```
man = []
other = []
try:
    data = open('sketch.txt')
    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)
```

Assign the stripped string back onto itself.

```
            line_spoken = line_spoken.strip()
```

The "strip()" method removes unwanted whitespace from a string.

```
            if role == 'Man':
                man.append(line_spoken)
            elif role == 'Other Man':
                other.append(line_spoken)
```

"elif" means "else if."

Update one of the lists based on who said what.

```
        except ValueError:
            pass
    data.close()
except IOError:
    print('The datafile is missing!')
```

```
print(man)
print(other)
```

Conclude by displaying the processed data on screen.

# Test Drive

Load your code into IDLE's edit window and take it for a spin by pressing F5. Be sure to save your program into the same folder that contains `sketch.txt`.

2-open-into-lists-cleaned-up.py - /Users/barryp/HeadFirstPython/chapter4/2-o...

```
man = []
other = []

try:
    data = open('sketch.txt')

    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)
            line_spoken = line_spoken.strip()
            if role == 'Man':
                man.append(line_spoken)
            elif role == 'Other Man':
                other.append(line_spoken)
        except ValueError:
            pass

    data.close()
except IOError:
    print('The datafile is missing!')

print(man)
print(other)
```

The code in IDLE's edit window

Python Shell

```
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ================================ RESTART ================================
>>>
['Is this the right room for an argument?', "No you haven't!", 'When?', "No yo
u didn't!", "You didn't!", 'You did not!', 'Ah! (taking out his wallet and pay
ing) Just the five minutes.', 'You most certainly did not!', "Oh no you didn't
!", "Oh no you didn't!", "Oh look, this isn't an argument!", "No it isn't!", "
It's just contradiction!", 'It IS!', 'You just contradicted me!', 'You DID!',
'You did just then!', '(exasperated) Oh, this is futile!!', 'Yes it is!']
["I've told you once.", 'Yes I have.', 'Just now.', 'Yes I did!', "I'm telling
you, I did!", "Oh I'm sorry, is this a five minute argument, or the full half
hour?", 'Just the five minutes. Thank you.', 'Anyway, I did.', "Now let's get
one thing quite clear: I most definitely told you!", 'Oh yes I did!', 'Oh yes
I did!', 'Yes it is!', "No it isn't!", 'It is NOT!', "No I didn't!", 'No no no
!', 'Nonsense!', "No it isn't!"]
>>>
```

Ln: 8 Col: 4

And here's what appears on screen: the contents of the two lists.

**It worked, as expected.**

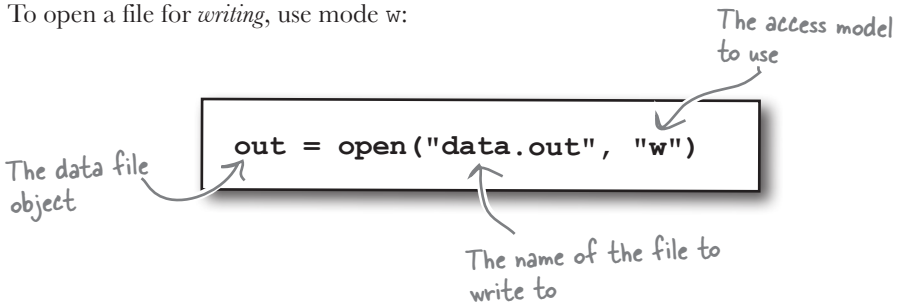
Surely Python's open() BIF can open files for writing as well as reading, eh?

**Yes, it can.**

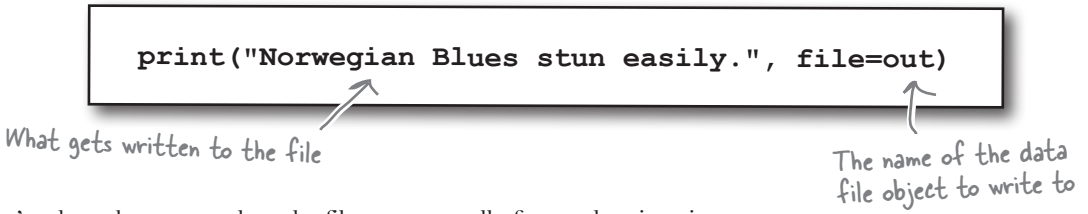
When you need to **save** data to a file, the `open()` BIF is all you need.

# Open your file in write mode

When you use the `open()` BIF to work with a disk file, you can specify an *access mode* to use. By default, `open()` uses mode `r` for *reading*, so you don't need to specify it. To open a file for *writing*, use mode `w`:




By default, the `print()` BIF uses standard output (usually the screen) when displaying data. To write data to a file instead, use the `file` argument to specify the data file object to use:




When you're done, be sure to close the file to ensure all of your data is written to disk. This is known as **flushing** and is very important:


## Geek Bits

When you use access mode `w`, Python opens your named file for **writing**. If the file already exists, it is *cleared of its contents*, or *clobbered*. To **append** to a file, use access mode `a`, and to open a file for writing and reading (without clobbering), use `w+`. If you try to open a file for writing that does not already exist, it is first created for you, and then opened for writing.

## Sharpen your pencil

At the bottom of your program, two calls to the `print()` BIF display your processed data on screen. Let's amend this code to save the data to two disk files instead.

Call your disk files `man_data.txt` (for what the man said) and `other_data.txt` (for what the other man said). Be sure to both **open** and **close** your data files, as well as protect your code against an `IOError` using **try/except**.

```
man = []
other = []

try:
    data = open('sketch.txt')
    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)
            line_spoken = line_spoken.strip()
            if role == 'Man':
                man.append(line_spoken)
            elif role == 'Other Man':
                other.append(line_spoken)
        except ValueError:
            pass
    data.close()
except IOError:
    print('The datafile is missing!')
```

Go on, try.

```
.........
    ..........................................................................
    ..........................................................................

    print(man, ..........................)
    print(other, ..........................)

.....
    ..................................
    ..................................

..........................
    ..................................
```

Open your two data files here.

Specify the files to write to when you invoke "print()".

Be sure to close your files.

Handle any exceptions here.

## Sharpen your pencil Solution

At the bottom of your program, two calls to the `print()` BIF display your processed data on screen. You were to amend this code to save the data to two disk files instead.

You were to call your disk files `man_data.txt` (for what the man said) and `other_data.txt` (for what the other man said). You were to make sure to both **open** and **close** your data files, as well as protect your code against an `IOError` using **try/except**.

```
man = []
other = []

try:
    data = open('sketch.txt')
    for each_line in data:
        try:
            (role, line_spoken) = each_line.split(':', 1)
            line_spoken = line_spoken.strip()
            if role == 'Man':
                man.append(line_spoken)
            elif role == 'Other Man':
                other.append(line_spoken)
        except ValueError:
            pass
    data.close()
except IOError:
    print('The datafile is missing!')
```

All of this code is unchanged.

Did you remember to open your files in WRITE mode?

```
try:
    man_file = open('man_data.txt', 'w')
    other_file = open('other_data.txt', 'w')

    print(man, file=man_file)
    print(other, file=other_file)

    man_file.close()
    other_file.close()
except IOError:
    print('File error.')
```

Open your two files, and assign each to file objects.

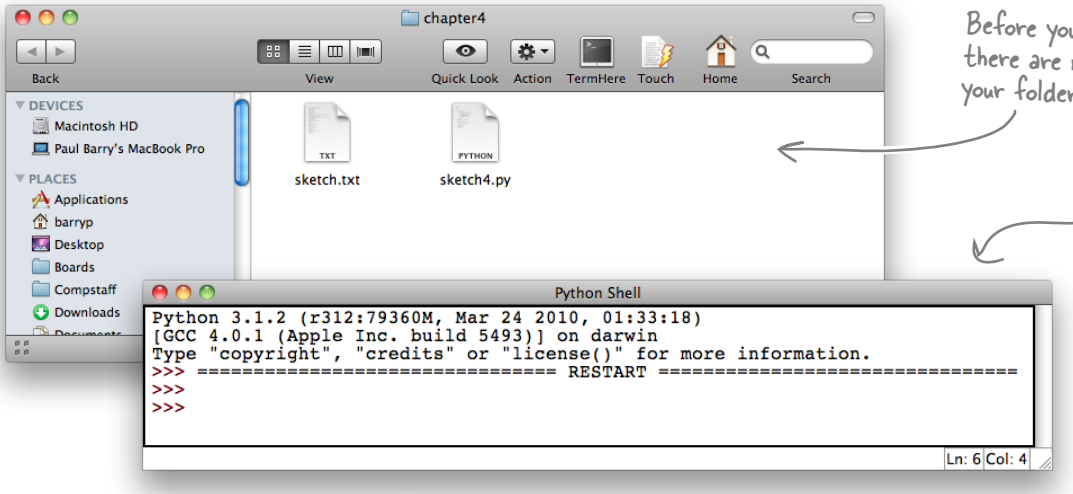
Use the "print()" BIF to save the named lists to named disk files.

Don't forget to close BOTH files.

Handle an I/O exception, should one occur.

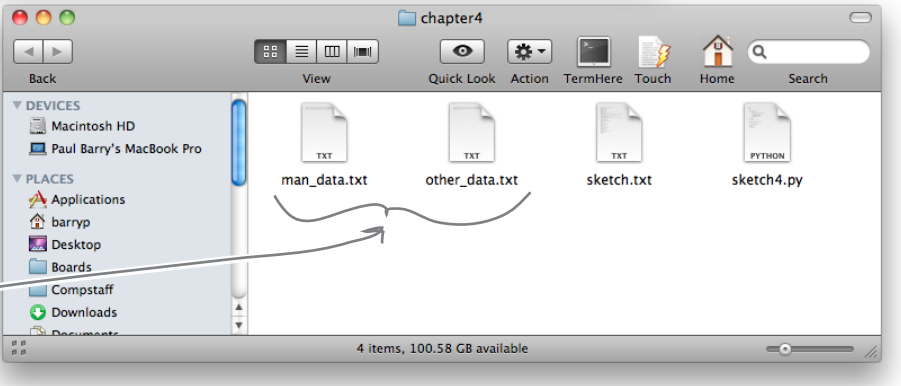
# Test Drive

Perform the edits to your code to replace your two `print()` calls with your new file I/O code. Then, run your program to confirm that the data files are created:




Before your program runs, there are no data files in your folder, just your code.

When you run your code, this is all you see, what looks like an "empty" IDLE shell.




After your program runs, two new files are CREATED in your folder.

That code worked, too. You've created two data files, each holding the data from each of your lists. Go ahead and open these files in your favorite editor to confirm that they contain the data you expect.

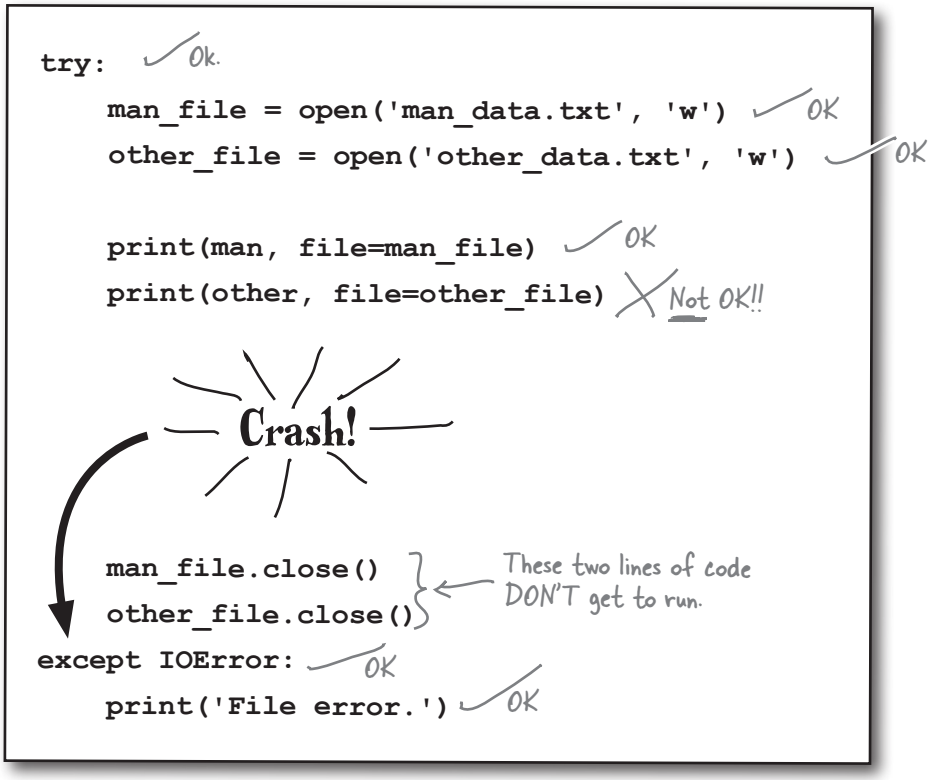
**Brain Power**

Consider the following carefully: what happens to your data files if the **second** call to `print()` in your code causes an `IOError`?

# Files are left open after an exception!

When all you ever do is read data from files, getting an `IOError` is annoying, but rarely dangerous, because your data is still in your file, even though you might be having trouble getting at it.

It's a different story when writing data to files: if you need to handle an `IOError` *before* a file is closed, your written data might become corrupted and there's no way of telling until *after* it has happened.

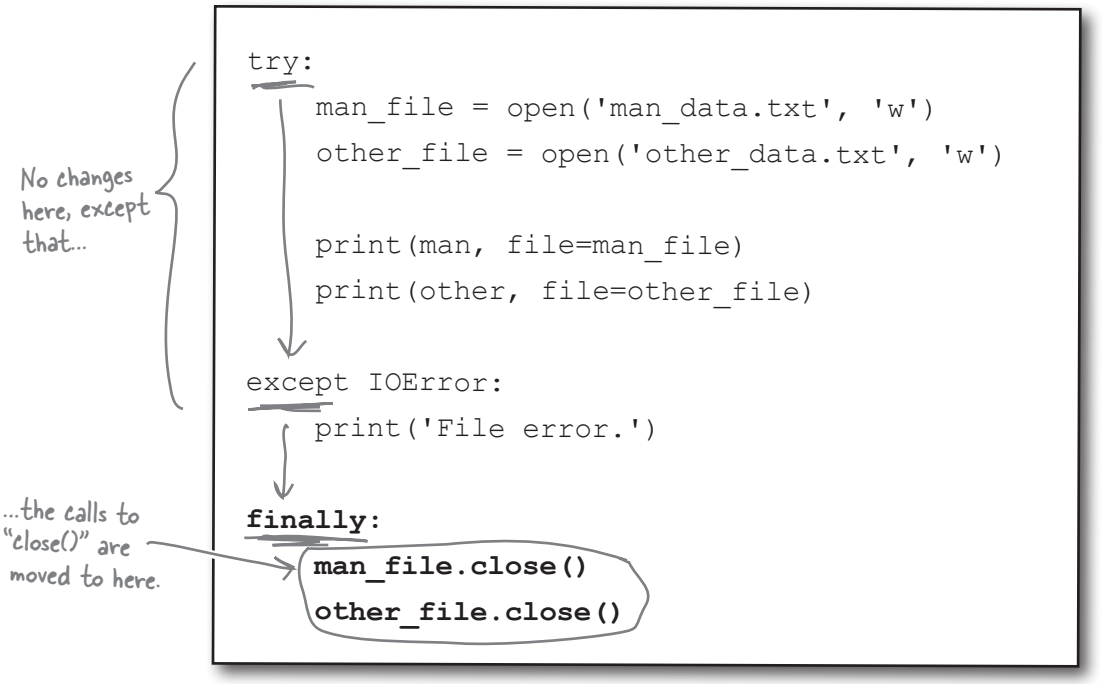



Your exception-handling code is doing its job, but you now have a situation where your data could *potentially* be corrupted, which can't be good.

What's needed here is something that lets you run some code regardless of whether an `IOError` has occured. In the context of your code, you'll want to make sure the files are closed *no matter what*.

# Extend try with finally

When you have a situation where code must *always* run no matter what errors occur, add that code to your `try` statement's `finally` suite:




If no runtime errors occur, any code in the `finally` suite executes. Equally, if an `IOError` occurs, the `except` suite executes and *then* the `finally` suite runs.

No matter what, the code in the finally suite *always* runs.

By moving your file closing code into your `finally` suite, you are reducing the possibility of data corruption errors.

This is a big improvement, because you're now ensuring that files are closed properly (even when write errors occur).

But what about *those* errors?

### How do you find out the specifics of the error?

## there are no Dumb Questions

**Q: I'm intrigued. When you stripped the line_spoken data of unwanted whitespace, you assigned the result back to the line_spoken variable. Surely invoking the strip() method on line_spoken changed the string it refers to?**

A: No, that's not what happens. Strings in Python are *immutable*, which means that once a string is created, it **cannot** be changed.

**Q: But you did change the line_spoken string by removing any unwanted whitespace, right?**

A: Yes and no. What actually happens is that invoking the `strip()` method on the `line_spoken` string creates a new string with leading and trailing whitespace removed. The new string is then assigned to `line_spoken`, replacing the data that was referred to before. In effect, it is as if you changed `line_spoken`, when you've actually completely replaced the data it refers to.

**Q: So what happens to the replaced data?**

A: Python's built-in memory management technology reclaims the RAM it was using and makes it available to your program. That is, unless some other Python data object is also referring to the string.

**Q: What? I don't get it.**

A: It is conceivable that another data object is referring to the string referred to by `line_spoken`. For example, let's assume you have some code that contains two variables that refer to the same string, namely "Flying Circus." You then decide that one of the variables needs to be in all UPPERCASE, so you invoke the `upper()` method on it. The Python interperter takes a copy of the string, converts it to uppercase, and returns it to you. You can then assign the uppercase data back to the variable that used to refer to the original data.

**Q: And the original data cannot change, because there's another variable referring to it?**

A: Precisely. That's why strings are immutable, because you never know what other variables are referring to any particular string.

**Q: But surely Python can work out how many variables are referring to any one particular string?**

A: It does, but only for the purposes of garbage collection. If you have a line of code like `print('Flying Circus')`, the string is not referred to by a variable (so any variable reference counting that's going on isn't going to count it) but is still a valid string object (which might be referred to by a variable) and it cannot have its data changed under any circumstances.

**Q: So Python variables don't actually contain the data assigned to them?**

A: That's correct. Python variables contain a reference to a data object.The data object contains the data and, because you can conceivably have a string object used in many different places throughout your code, it is safest to make all strings immutable so that no nasty side effects occur.

**Q: Isn't it a huge pain not being able to adjust strings "in place"?**

A: No, not really. Once you get used to how strings work, it becomes less of an issue. In practice, you'll find that this issue rarely trips you up.

**Q: Are any other Python data types immutable?**

A: Yes, a few. There's the tuple, which is an immutable list. Also, all of the number types are immutable.

**Q: Other than learning which is which, how will I know when something is immutable?**

A: Don't worry: you'll know. If you try to change an immutable value, Python raises a `TypeError` exception.

**Q: Of course: an exception occurs. They're everywhere in Python, aren't they?**

A: Yes. Exceptions make the world go 'round.

# Knowing the type of error is not enough

When a file I/O error occurs, your code displays a generic "File Error" message. This is too generic. How do you know what actually happened?


### Who knows?

It turns out that the Python interpreter knows…and it will give up the details if only you'd ask.

When an error occurs at runtime, Python raises an exception of the specific type (such as `IOError`, `ValueError`, and so on). Additionally, Python creates an **exception object** that is passed *as an argument* to your **except** suite.

### Let's use IDLE to see how this works.

## An IDLE Session

Let's see what happens when you try to open a file that doesn't exist, such as a disk file called `missing.txt`. Enter the following code at IDLE's shell:

```
>>> try:
        data = open('missing.txt')
        print(data.readline(), end='')
except IOError:
        print('File error')
finally:
        data.close()
```

```
File error
Traceback (most recent call last):
  File "<pyshell#8>", line 7, in <module>
    data.close()
NameError: name 'data' is not defined
```

There's your error message, but...

...what's this?!? Another exception was raised and it killed your code.

As the file doesn't exist, the `data` file object **wasn't** created, which subsequently makes it impossible to call the `close()` method on it, so you end up with a `NameError`. A quick fix is to add a small test to the `finally` suite to see if the `data` name exists *before* you try to call `close()`. The `locals()` BIF returns a collection of names defined in the current scope. Let's exploit this BIF to only invoke `close()` when it is safe to do so:

```
finally:
        if 'data' in locals():
            data.close()
```

The "in" operator tests for membership.

This is just the bit of code that needs to change. Press Alt-P to edit your code at IDLE's shell.

```
File error
```

No extra exceptions this time. Just your error message.

Here you're searching the collection returned by the `locals()` BIF for the string `data`. If you find it, you can assume the file was opened successfully and safely call the `close()` method.

If some other error occurs (perhaps something awful happens when your code calls the `print()` BIF), your exception-handling code **catches** the error, **displays** your "File error" message and, finally, **closes** any opened file.

But you still are none the wiser as to *what actually caused the error*.

When an exception is raised and handled by your `except` suite, the Python interpreter passes an *exception object* into the suite. A small change makes this exception object available to your code as an identifier:

```
except IOError as err:
        print('File error: ' + err)
```

Give your exception object a name...

...then use it as part of your error message.

But when you try to run your code with this change made, another exception is raised:

```
Traceback (most recent call last):
  File "<pyshell#18>", line 5, in <module>
    print('File error:' + err)
TypeError: Can't convert 'IOError' object to str implicitly
```

Whoops! Yet another exception; this time it's a "TypeError".

This time your error message didn't appear *at all*. It turns out exception objects and strings are not *compatible* types, so trying to concatenate one with the other leads to problems. You can convert (or *cast*) one to the other using the `str()` BIF:

```
except IOError as err:
        print('File error: ' + str(err))
```

Use the "str()" BIF to force the exception object to behave like a string.

Now, with this final change, your code is behaving exactly as expected:

```
File error: [Errno 2] No such file or directory: 'missing.txt'
```

And you now get a specific error message that tells you exactly what went wrong.

**Of course, all this extra logic is starting to obscure the real meaning of your code.**

Wouldn't it be dreamy if there were a way to take advantage of these mechanisms without the code bloat? I guess it's just a fantasy...

# Use with to work with files

Because the use of the **try/except/finally** pattern is so common when it comes to working with files, Python includes a statement that abstracts away some of the details. The `with` statement, when used with files, can dramatically reduce the amount of code you have to write, because it negates the need to include a `finally` suite to handle the closing of a *potentially* opened data file. Take a look:

```
try:
    data = open('its.txt', "w")
    print("It's...", file=data)
except IOError as err:
    print('File error: ' + str(err))
finally:
    if 'data' in locals():
        data.close()
```

This is the usual "try/except/finally" pattern.

```
try:
    with open('its.txt', "w") as data:
        print("It's...", file=data)
except IOError as err:
    print('File error: ' + str(err))
```

The use of "with" negates the need for the "finally" suite.

When you use `with`, you no longer have to worry about closing any opened files, as the Python interpreter automatically takes care of this for you. The `with` code on the the right is *identical* in function to that on the left. At Head First Labs, we know which approach we prefer.

## Geek Bits

The with statement takes advantage of a Python technology called the **context management protocol**.

## Sharpen your pencil

Grab your pencil and rewrite this **try/except/finally** code to use `with` instead. Here's your code with the appropriate `finally` suite added:

```
try:
    man_file = open('man_data.txt', 'w')
    other_file = open('other_data.txt', 'w')

    print(man, file=man_file)
    print(other, file=other_file)
except IOError as err:
    print('File error: ' + str(err))
finally:
    if 'man_file' in locals():
        man_file.close()
    if 'other_file' in locals():
        other_file.close()
```

Write your "with" code here.

## Sharpen your pencil Solution

You were to grab your pencil and rewrite this **try/except/finally** code to use `with` instead. Here's your code with the appropriate `finally` suite added:

```
try:
    man_file = open('man_data.txt', 'w')
    other_file = open('other_data.txt', 'w')

    print(man, file=man_file)
    print(other, file=other_file)
except IOError as err:
    print('File error: ' + str(err))
finally:
    if 'man_file' in locals():
        man_file.close()
    if 'other_file' in locals():
        other_file.close()
```

Using two "with" statements to rewrite the code without the "finally" suite.

```
try:
    with open('man_data.txt', 'w') as man_file:
        print(man, file=man_file)
    with open('other_data.txt', 'w') as other_file:
        print(other, file=other_file)
except IOError as err:
    print('File error: ' + str(err))
```

Or combine the two "open()" calls into one "with" statement.

Note the use of the comma.

```
with open('man_data.txt', 'w') as man_file, open('other_data.txt', 'w') as other_file:
    print(man, file=man_file)
    print(other, file=other_file)
```

# Test Drive

Add your `with` code to your program, and let's confirm that it continues to function as expected. Delete the two data files you created with the previous version of your program and then load your newest code into IDLE and give it a spin.

No errors in the IDLE shell appears to indicate that the program ran successfully.

```
Python Shell
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ================================ RESTART ================================
>>>
>>>
                                                                  Ln: 6 Col: 4
```

If you check your folder, your two data files should've reappeared. Let's take a closer look at the data file's contents by opening them in your favorite text editor (or use IDLE).

```
man_data.txt + (~/HeadFirstPython/chapter4) - VIM3
['Is this the right room for an argument?', "No you haven't!", 'When?',
 "No you didn't!", "You didn't!", 'You did not!', 'Ah! (taking out his
wallet and paying) Just the five minutes.', 'You most certainly did not
!', "Oh no you didn't!", "Oh no you didn't!", "Oh look, this isn't an a
rgument!", "No it isn't!", "It's just contradiction!", 'It IS!', 'You j
ust contradicted me!', 'You DID!', 'You did just then!', '(exasperated)
 Oh, this is futile!!', 'Yes it is!']
~
~
~
~
~
~
~
```

Here's what the man said.

```
other_data.txt + (~/HeadFirstPython/chapter4) - VIM3
["I've told you once.", 'Yes I have.', 'Just now.', 'Yes I did!', "I'm
telling you, I did!", "Oh I'm sorry, is this a five minute argument, or
 the full half hour?", 'Just the five minutes. Thank you.', 'Anyway, I
did.', 'Oh yes I did!', 'Oh yes I did!', 'Yes it is!', "No it isn't!",
'It is NOT!', "No I didn't!", 'No no no!', 'Nonsense!', "No it isn't!"]
~
~
~
~
~
~
```

Here's what the other man said.

You've saved the lists in two files containing what the *Man* said and what the *Other man* said. Your code is smart enough to handle any exceptions that Python or your operating system might throw at it.

**Well done. This is really coming along.**

# Default formats are unsuitable for files

Although your data is now stored in a file, it's not really in a useful format. Let's experiment in the IDLE shell to see what impact this can have.

## An IDLE Session

Use a `with` statement to open your data file and display a single line from it:

```
>>> with open('man_data.txt') as mdf:
        print(mdf.readline())

['Is this the right room for an argument?', "No you haven't!", 'When?', "No you didn't!", "You
didn't!", 'You did not!', 'Ah! (taking out his wallet and paying) Just the five minutes.',
'You most certainly did not!', "Oh no you didn't!", "Oh no you didn't!", "Oh look, this isn't
an argument!", "No it isn't!", "It's just contradiction!", 'It IS!', 'You just contradicted
me!', 'You DID!', 'You did just then!', '(exasperated) Oh, this is futile!!', 'Yes it is!']
```

Note: no need to close your file, because "with" does that for you.

Yikes! It would appear your list is converted to a large string by `print()` when it is saved. Your experimental code reads a single line of data from the file and gets *all* of the data as one large chunk of text…so much for your code saving your *list* data.

**What are your options for dealing with this problem?**

## Geek Bits

By default, `print()` displays your data in a format that mimics how your list data is actually stored by the Python interpreter. The resulting output is not really meant to be processed further… its primary purpose is to show you, the Python programmer, what your list data "looks like" in memory.


Parsing the data in the file is a possibility…although it's *complicated* by all those square brackets, quotes, and commas. Writing the required code is doable, but it is a lot of code just to read back in your saved data.

Of course, if the data is in a *more easily parseable format*, the task would likely be easier, so maybe the second option is worth considering, too?


Can you think of a function you created from earlier in this book that might help here?

# Why not modify print_lol()?

Recall your `print_lol()` function from Chapter 2, which takes any list (or list of lists) and displays it on screen, one line at a time. And nested lists can be indented, if necessary.

This functionality sounds perfect! Here's your code from the `nester.py` module (last seen at the end of Chapter 2):

nester.py – /Users/barryp/Downloads/nester-1.3.0/nester.py

```
"""This is the "nester.py" module and it provides one function called print_lol()
   which prints lists that may or may not include nested lists."""

def print_lol(the_list, indent=False, level=0):
    """ Prints a list of (possibly) nested lists.

        This function takes a positional argument called "the_list", which
        is any Python list (of - possibly - nested lists). Each data item in the
        provided list is (recursively) printed to the screen on it's own line.
        A second argument called "indent" controls whether or not indentation is
        shown on the display. This defaults to False: set it to True to switch on.
        A third argument called "level" (which defaults to 0) is used to insert
        tab-stops when a nested list is encountered."""

    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item, indent, level+1)
        else:
            if indent:
                for tab_stop in range(level):
                    print("\t", end='')
            print(each_item)
```

This code currently displays your data on the screen.

Ln: 24 Col: 0

Amending this code to print to a disk file instead of the screen (known as *standard output*) should be relatively straightforward. You can then save your data in a more usable format.

## the Scholar's Corner

**Standard Output** The default place where your code writes its data when the "print()" BIF is used. This is typically the screen. In Python, standard output is referred to as "sys.stdout" and is importable from the Standard Library's "sys" module.


1. Let's add a fourth argument to your `print_lol()` function to identify a place to write your data to. Be sure to give your argument a default value of `sys.stdout`, so that it continues to write to the screen if no file object is specified when the function is invoked.

   Fill in the blanks with the details of your new argument. (**Note:** to save on space, the comments have been removed from this cod, but be sure to update your comments in your `nester.py` module after you've amended your code.)

```
def print_lol(the_list, indent=False, level=0,.....):

    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item, indent, level+1,.....)
        else:
            if indent:
                for tab_stop in range(level):
                    print("\t", end='',.....)
            print(each_item,.....)
```

2. What needs to happen to the code in your `with` statement now that your amended `print_lol()` function is available to you?

   .....

   .....

   .....

3. List the name of the module(s) that you now need to import into your program in order to support your amendments to `print_lol()`.

   .....

   .....


1. You were to add a fourth argument to your `print_lol()` function to identify a place to write your data to, being sure to give your argument a default value of `sys.stdout` so that it continues to write to the screen if no file object is specified when the function is invoked.

   You were to fill in the blanks with the details of your new argument. (**Note:** to save on space, the comments have been removed from this code, but be sure to update those in your `nester.py` module after you've amended your code).

Add the fourth argument and give it a default value.

```
def print_lol(the_list, indent=False, level=0, fh=sys.stdout ):

    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item, indent, level+1, fh )
        else:
            if indent:
                for tab_stop in range(level):
                    print("\t", end='', file=fh )
            print(each_item, file=fh )
```

Note: the signature has changed.

Adjust the two calls to "print()" to use the new argument.

2. What needs to happen to the code in your `with` statement now that your amended `print_lol()` function is available to you?

The code needs to be adjusted so that instead of using the "print()" BIF, the code needs to invoke "print_lol()" instead.

3. List the name of the module(s) that you now need to import into your program in order to support your amendments to `print_lol()`.

The program needs to import the amended "nester" module.

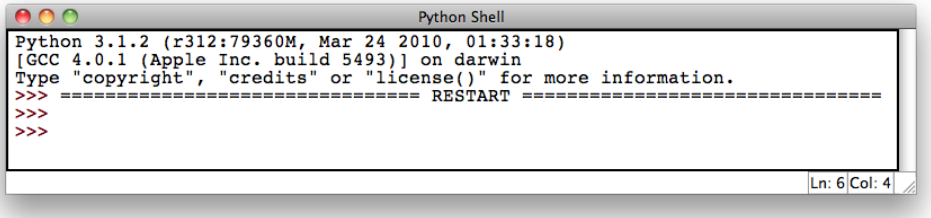
# Test Drive

Before taking your code for a test drive, you need to do the following:

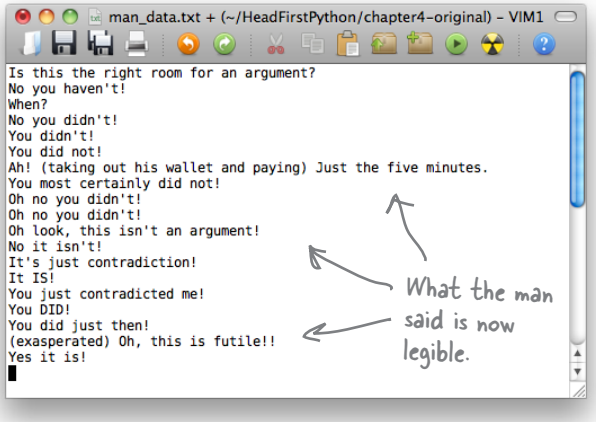
1. Make the necessary changes to `nester` and install the amended module into your Python environment (see Chapter 2 for a refresher on this). You might want to upload to PyPI, too.
2. Amend your program so that it imports `nester` and uses `print_lol()` instead of `print()` within your **with** statement. **Note:** your `print_lol()` invocation should look something like this: `print_lol(man, fh=man_file)`.

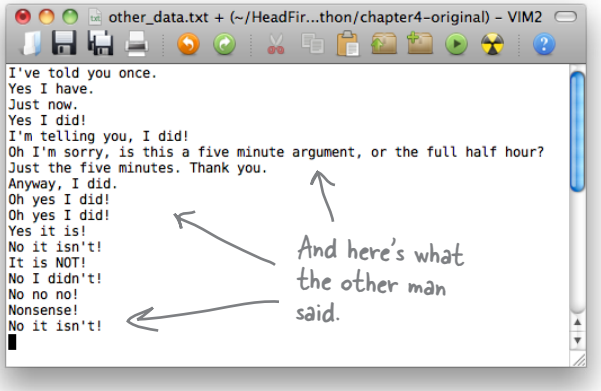
When you are ready, take your latest program for a test drive and let's see what happens:

As before, there's no output on screen.



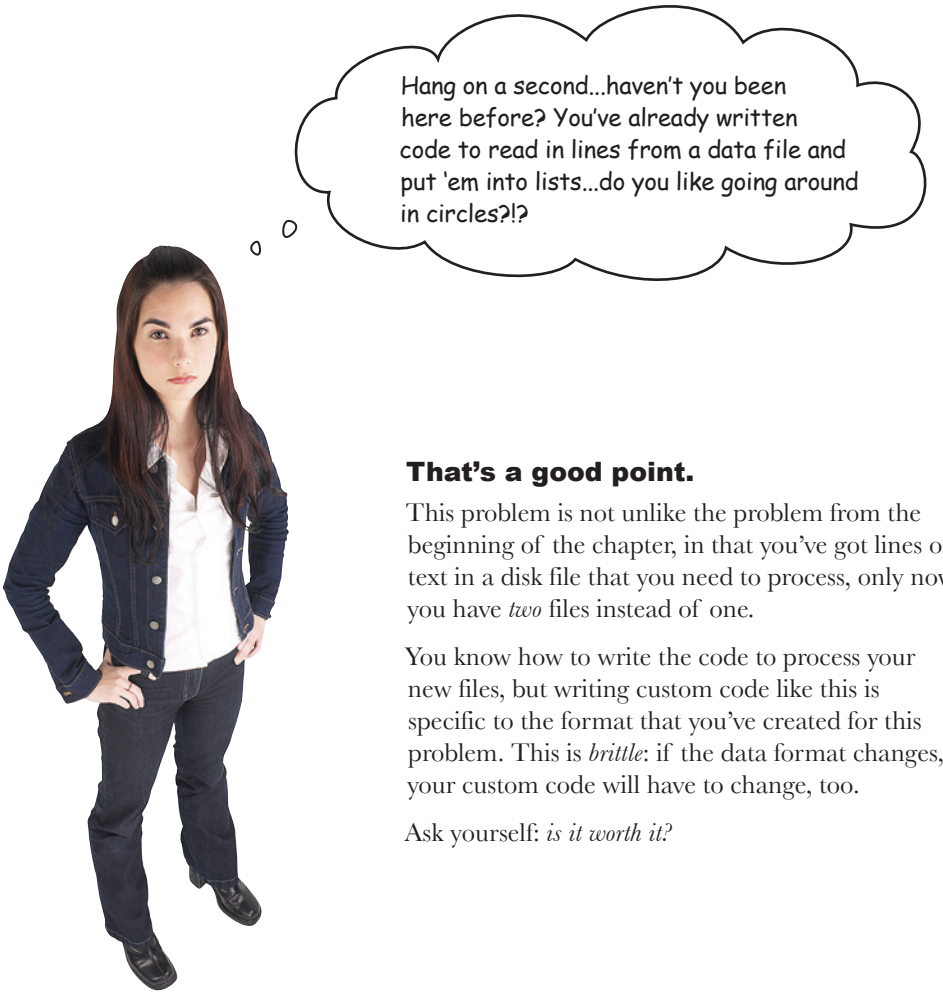

Let's check the contents of the files to see what they look like now.




This is looking good. By amending your `nester` module, you've provided a facility to save your list data in a legible format. It's now way easier on the eye.

**But does this make it any easier to read the data back in?**




**That's a good point.**

This problem is not unlike the problem from the beginning of the chapter, in that you've got lines of text in a disk file that you need to process, only now you have *two* files instead of one.

You know how to write the code to process your new files, but writing custom code like this is specific to the format that you've created for this problem. This is *brittle*: if the data format changes, your custom code will have to change, too.

Ask yourself: *is it worth it?*

## Custom Code Exposed

**This week's interview:**
**When is custom code appropriate?**

**Head First:** Hello, CC, how are you today?

**Custom Code:** Hi, I'm great! And when I'm not great, there's always something I can do to fix things. Nothing's too much trouble for me. Here: have a seat.

**Head First:** Why, thanks.

**Custom Code:** Let me get that for you. It's my new custom *SlideBack&Groove™*, the 2011 model, with added cushions and lumbar support…and it automatically adjusts to your body shape, too. How does that feel?

**Head First:** Actually [relaxes], that feels kinda groovy.

**Custom Code:** See? Nothing's too much trouble for me. I'm your "go-to guy." Just ask; absolutely anything's possible when it's a custom job.

**Head First:** Which brings me to why I'm here. I have a "delicate" question to ask you.

**Custom Code:** Go ahead, shoot. I can take it.

**Head First:** When is custom code appropriate?

**Custom Code:** Isn't it obvious? It's *always* appropriate.

**Head First:** Even when it leads to problems down the road?

**Custom Code:** Problems?!? But I've already told you: nothing's too much trouble for me. I live to customize. If it's broken, I fix it.

**Head First:** Even when a readymade solution might be a better fit?

**Custom Code:** Readymade? You mean (I hate to say it): *off the shelf*?

**Head First:** Yes. Especially when it comes to writing complex programs, right?

**Custom Code:** What?!? That's where I excel: creating beautifully crafted custom solutions for folks with complex computing problems.

**Head First:** But if something's been done before, why reinvent the wheel?

**Custom Code:** But everything I do is custom-made; that's why people come to me…

**Head First:** Yes, but if you take advantage of other coders' work, you can build your own stuff in half the time *with less code*. You can't beat that, can you?

**Custom Code:** "Take advantage"…isn't that like *exploitation*?

**Head First:** More like collaboration, sharing, participation, *and* working together.

**Custom Code:** [shocked] You want me to give my code…*away*?

**Head First:** Well…more like share and share alike. I'll scratch your back if you scratch mine. How does that sound?

**Custom Code:** That sounds disgusting.

**Head First:** Very droll [laughs]. All I'm saying is that it is not always a good idea to create everything from scratch with custom code when a good enough solution to the problem might already exist.

**Custom Code:** I guess so…although it won't be as perfect a fit as that chair.

**Head First:** But I will be able to sit on it!

**Custom Code:** [laughs] You should talk to my buddy **Pickle**…he's forever going on about stuff like this. And to make matters worse, he lives in a library.

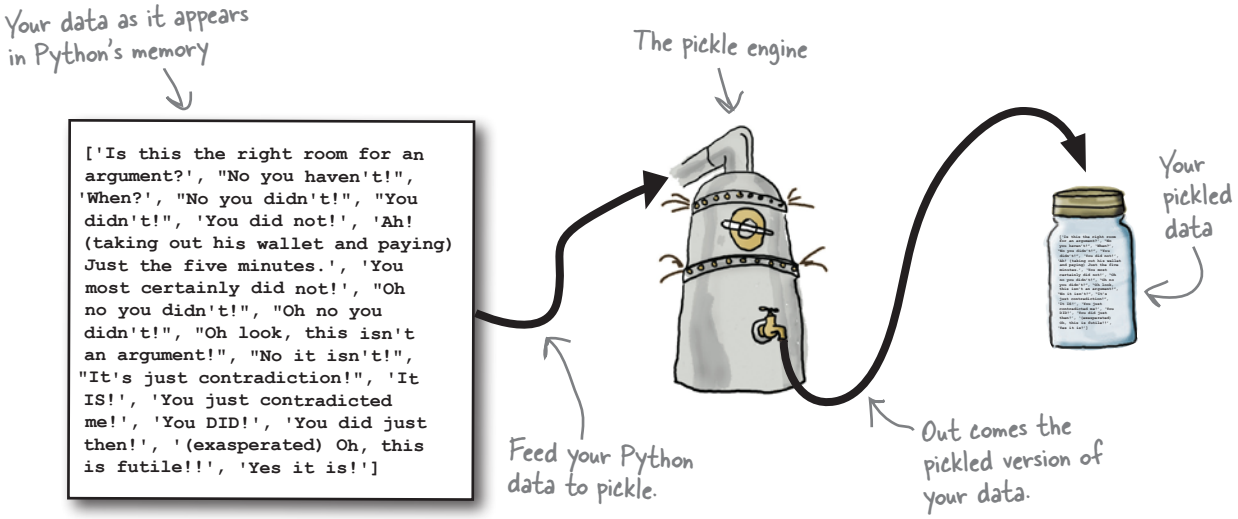
**Head First:** I think I'll give him a shout. Thanks!

**Custom Code:** Just remember: you know where to find me if you need any custom work done.

# Pickle your data

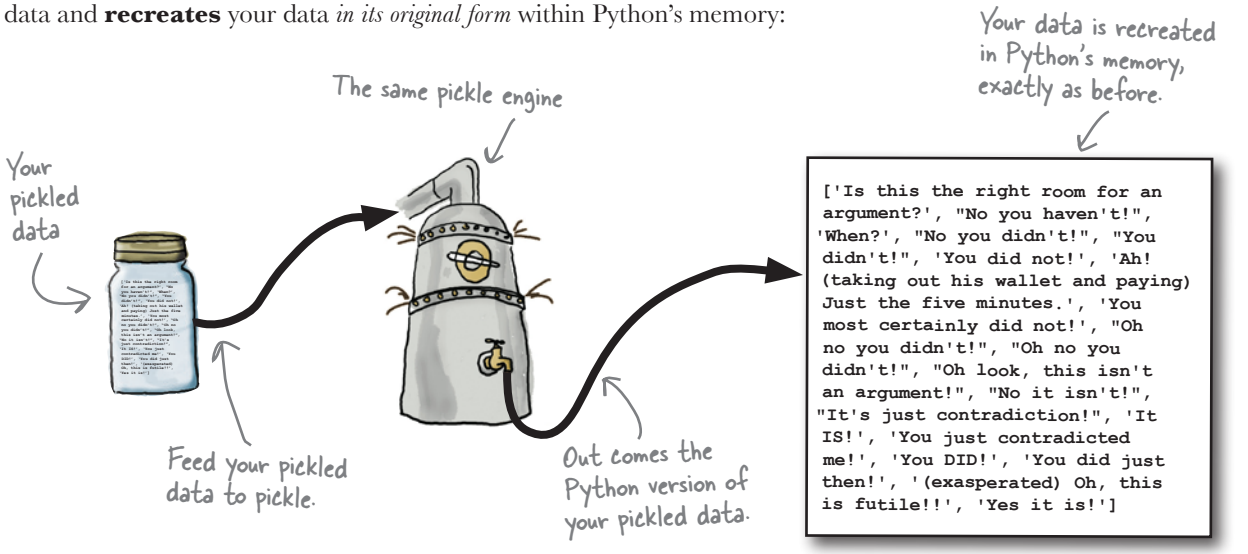
Python ships with a standard library called `pickle`, which can save and load almost any Python data object, including lists.

Once you *pickle* your data to a file, it is **persistent** and ready to be read into another program at some later date/time:



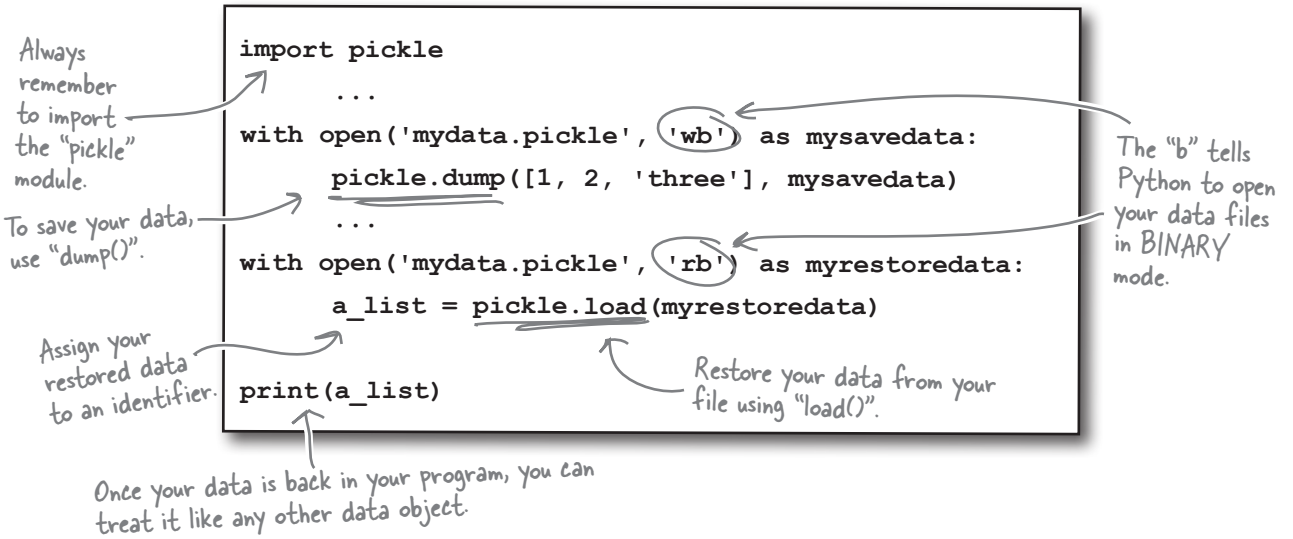

You can, for example, **store** your pickled data on disk, **put** it in a database, or **transfer** it over a network to another computer.

When you are ready, reversing this process unpickles your persistent pickled data and **recreates** your data *in its original form* within Python's memory:



# Save with dump and restore with load

Using `pickle` is straightforward: import the required module, then use `dump()` to save your data and, some time later, `load()` to restore it. The only requirement when working with pickled files is that they have to be opened in *binary access mode*:




```
import pickle
      ...
with open('mydata.pickle', 'wb') as mysavedata:
      pickle.dump([1, 2, 'three'], mysavedata)
      ...
with open('mydata.pickle', 'rb') as myrestoredata:
      a_list = pickle.load(myrestoredata)

print(a_list)
```

## What if something goes wrong?

If something goes wrong when pickling or unpickling your data, the `pickle` module raises an exception of type `PickleError`.

### Sharpen your pencil

Here's a snippet of your code as it currently stands. Grab your pencil and strike out the code you no longer need, and then replace it with code that uses the facilities of `pickle` instead. Add any additional code that you think you might need, too.

```
try:
    with open('man_data.txt', 'w') as man_file, open('other_data.txt', 'w') as other_file:
        nester.print_lol(man, fh=man_file)
        nester.print_lol(other, fh=other_file)
except IOError as err:
    print('File error: ' + str(err))
```

## Sharpen your pencil Solution

Here's a snippet of your code as it currently stands. You were to grab your pencil and strike out the code you no longer need, and then replace it with code that uses the facilities `pickle` instead. You were also to add any additional code that you think you might need.

Import "pickle" near the top of your program.

```
import pickle

try:
    with open('man_data.txt', 'w') as man_file, open('other_data.txt', 'w') as other_file:
        nester.print_lol(man, fh=man_file)
        nester.print_lol(other, fh=other_file)
except IOError as err:
    print('File error: ' + str(err))
except pickle.PickleError as perr:
    print('Pickling error: ' + str(perr))
```

Change the access mode to be "writeable, binary".

'wb'

'wb'

pickle.dump(man, man_file)

pickle.dump(other, other_file)

Replace the two calls to "nester.print_lol()" with calls to "pickle.dump()".

Don't forget to handle any exceptions that can occur.

## there are no Dumb Questions

**Q: When you invoked print_lol() earlier, you provided only two arguments, even though the function signature requires you to provide four. How is this possible?**

A: When you invoke a Python function in your code, you have options, especially when the function provides default values for some arguments. If you use positional arguments, the position of the argument in your function invocation dictates what data is assigned to which argument. When the function has arguments that also provide default values, you do not need to always worry about positional arguments being assigned values.

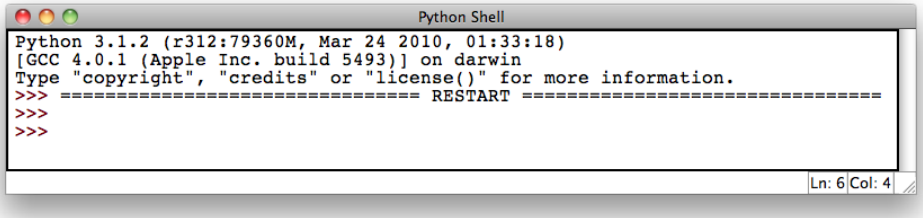
**Q: OK, you've completely lost me. Can you explain?**

A: Consider `print()`, which has this signature: `print(value, sep=' ', end='\n', file=sys.stdout)`. By default, this BIF displays to standard output (the screen), because it has an argument called `file` with a default value of `sys.stdout`. The file argument is the fourth positional argument. However, when you want to send data to something other than the screen, you do not need to (nor want to have to) include values for the second and third positional arguments. They have default values anyway, so you need to provide values for them only if the defaults are not what you want. If all you want to do is to send data to a file, you invoke the `print()` BIF like this: `print("Dead Parrot Sketch", file='myfavmonty.txt')` and the fourth positional argument uses the value you specify, while the other positional arguments use their defaults. In Python, not only do the BIFs work this way, but your custom functions support this mechamism, too.

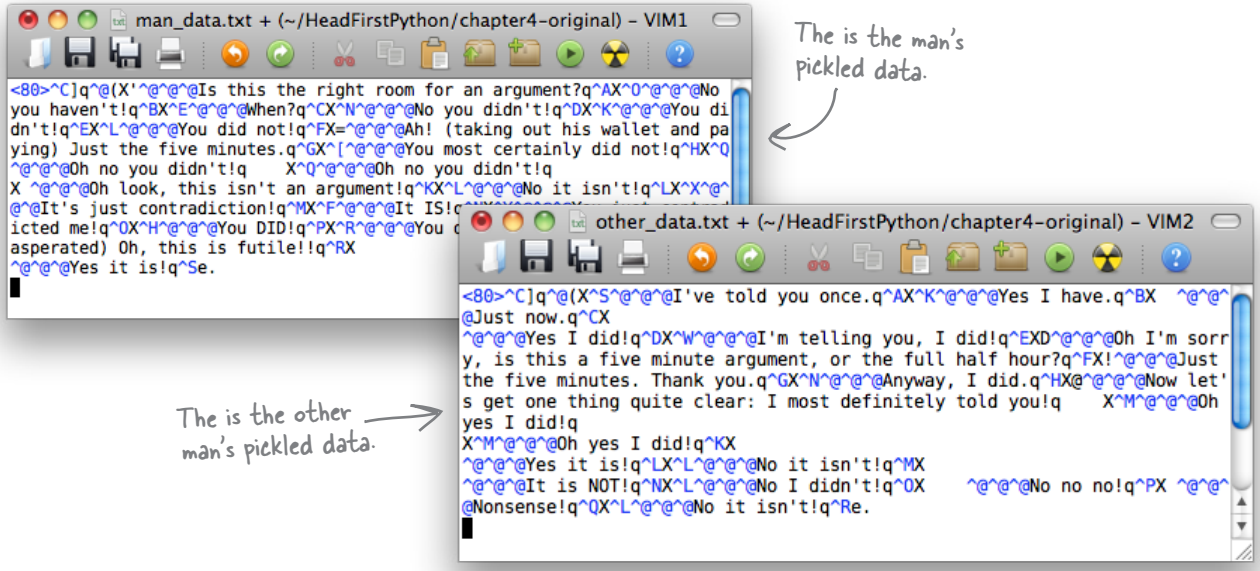
# Test Drive

Let's see what happens now that your code has been amended to use the standard `pickle` module instead of your custom `nester` module. Load your amended code into IDLE and press F5 to run it.

Once again, you get no visual clue that something has happened.




So, once again, let's check the contents of the files to see what they look like now:




It appears to have worked…but these files look like *gobbledygook*! What gives?

Recall that Python, not you, is pickling your data. To do so efficiently, Python's `pickle` module uses a custom binary format (known as its **protocol**). As you can see, viewing this format in your editor looks decidedly *weird*.

**Don't worry: it is supposed to look like this.**

## An IDLE Session

`pickle` really shines when you load some previously pickled data into another program. And, of course, there's nothing to stop you from using `pickle` with `nester`. After all, each module is designed to serve different purposes. Let's demonstrate with a handful of lines of code within IDLE's shell. Start by importing any required modules:

```
>>> import pickle
>>> import nester
```

No surprises there, eh?

Next up: create a new identifier to hold the data that you plan to unpickle.Create an empty list called `new_man`:

```
>>> new_man = []
```

Yes, almost too exciting for words, isn't it? With your list created. let's load your pickled data into it. As you are working with external data files, it's best if you enclose your code with **try/except**:

```
>>> try:
        with open('man_data.txt', 'rb') as man_file:
            new_man = pickle.load(man_file)
except IOError as err:
        print('File error: ' + str(err))
except pickle.PickleError as perr:
        print('Pickling error: ' + str(perr))
```

This code is not news to you either. However, at this point, your data has been unpickled and assigned to the `new_man` list. It's time for `nester` to do its stuff:

```
>>> nester.print_lol(new_man)
Is this the right room for an argument?
No you haven't!
When?
No you didn't!
   ...
You did just then!
(exasperated) Oh, this is futile!!
Yes it is!
```

*Not all the data is shown here, but trust us: it's all there.*

And to finish off, let's display the first line spoken as well as the last:

```
>>> print(new_man[0])
Is this the right room for an argument?
>>> print(new_man[-1])
Yes it is!
```

*See: after all that, it is the right room! ☺*

# Generic file I/O with pickle is the way to go!


Python takes care of your file I/O details, so you can concentrate on what your code actually does or needs to do.

As you've seen, being able to work with, save, and restore data in lists is a breeze, thanks to Python. But what other **data structures** does Python support *out of the box*?

**Let's dive into Chapter 5 to find out.**

# Your Python Toolbox

**You've got Chapter 4 under your belt and you've added some key Python techiques to your toolbox.**

## Python Lingo

- "Immutable types" – data types in Python that, once assigned a value, cannot have that value changed.
- "Pickling" – the process of saving a data object to persistence storage.
- "Unpickling" – the process of restoring a saved data object from persistence storage.

## BULLET POINTS

- The `strip()` method removes unwanted whitespace from strings.
- The `file` argument to the `print()` BIF controls where data is sent/saved.
- The `finally` suite is always executed no matter what exceptions occur within a **try/except** statement.
- An exception object is passed into the `except` suite and can be assigned to an identifier using the `as` keyword.
- The `str()` BIF can be used to access the stringed representation of any data object that supports the conversion.
- The `locals()` BIF returns a collection of variables within the current scope.
- The `in` operator tests for membership.
- The "+" operator concatenates two strings when used with strings but adds two numbers together when used with numbers.
- The `with` statement automatically arranges to close all opened files, even when exceptions occur. The `with` statement uses the `as` keyword, too.
- `sys.stdout` is what Python calls "standard output" and is available from the standard library's `sys` module.
- The standard library's `pickle` module lets you easily and efficiently save and restore Python data objects to disk.
- The `pickle.dump()` function saves data to disk.
- The `pickle.load()` function restores data from disk.